MERN STACK

INTERVIEW QUESTIONS & ANSWERS

JavaScript | Node.js | Express.js | React.js | MongoDB

English + Hinglish Explanations

Prepared for: Deepak Yadav

B.Tech CSE / AI-ML

190+ Questions | Complete Interview Preparation & Quick Revision

TABLE OF CONTENTS

PART 1 - JAVASCRIPT **4**

Basic JavaScript (Q1 - Q20) 4

Functions (Q21 - Q30) 14

Advanced JavaScript (Q31 - Q50) 19

PART 2 - NODE.JS **31**

Node.js (Q1 - Q30) 31

PART 3 - EXPRESS.JS **47**

Express.js (Q1 - Q30) 47

PART 4 - REACT.JS **63**

React.js (Q1 - Q30) 63

PART 5 - MONGODB + MONGOOSE **79**

MongoDB Basics (Q1 - Q12) 79

CRUD Operations and Queries (Q13 - Q27) 85

Mongoose (Q38 - Q50) 97

FINAL QUICK REVISION **105**

JavaScript Quick Revision 105

Node.js Quick Revision 105

Express.js Quick Revision 105

React.js Quick Revision 106

MongoDB Quick Revision 106

Most Important Differences 107

Interview Speaking Practice 109

PART 1

JAVASCRIPT

PART 1 - JAVASCRIPT

Basic JavaScript (Q1 - Q20)

Q1. What is JavaScript?

Easy English:

JavaScript is a high-level programming language used to make websites interactive.

- It runs inside the browser and also on the server using Node.js.
- It is used with HTML and CSS. HTML gives structure, CSS gives design, and JavaScript gives behaviour.
- It is a dynamically typed and single-threaded language.

Hinglish:

JavaScript ek programming language hai jo website ko interactive banati hai. HTML dhaancha deta hai, CSS design deta hai, aur JavaScript kaam (behaviour) karti hai. Ye browser me bhi chalti hai aur Node.js ki madad se server par bhi.

Example:

```
let name = "Deepak";
console.log("Hello " + name);   // Hello Deepak
```

Why is it used?

We use JavaScript to add logic to web pages, handle user clicks, call APIs, validate forms, and build full applications on both frontend and backend.

Interview Answer:

JavaScript is a high-level, dynamically typed programming language used to make web pages interactive. It runs in the browser and also on the server with Node.js, so we can build both frontend and backend with one language.

Important Point:

- JavaScript is single-threaded but handles async work using the event loop.
- JavaScript and Java are completely different languages.
- It follows the ECMAScript (ES) standard.

Q2. Why do we use JavaScript?

Easy English:

We use JavaScript because HTML and CSS alone cannot take decisions or react to a user.

- It handles events like click, scroll and typing.
- It can update the page content without reloading (DOM manipulation).
- It can talk to a server using API calls (fetch/axios).
- With Node.js we can also write backend code in the same language.

Hinglish:

JavaScript isliye use karte hain kyunki HTML aur CSS sirf page dikhate hain, kuch decide nahi kar sakte. Click, form validation, API call, page ko bina reload kiye update karna - ye sab JavaScript karti hai. Aur Node.js se backend bhi isi language me ban jata hai.

Example:

```
document.getElementById("btn").addEventListener("click", () => {
  alert("Button clicked!");
});
```

Why is it used?

It is used because one language covers frontend, backend, mobile and even desktop apps, which saves time and makes a developer productive.

Interview Answer:

We use JavaScript because it makes a website dynamic. It handles user events, updates the DOM, and connects to APIs. With Node.js the same language also works on the backend, which is the base of the MERN stack.

Important Point:

- JavaScript is the only language browsers run natively.
- MERN stack is fully JavaScript based.

Q3. What is console.log()?

Easy English:

console.log() is a built-in method that prints a message or value in the browser console or terminal.

- It is mainly used for debugging and checking values.
- It can print strings, numbers, arrays, objects and variables.

Hinglish:

console.log() ek built-in function hai jo value ko console me print karta hai. Iska main kaam debugging hai, matlab yeh dekhna ki code me value sahi aa rahi hai ya nahi.

Example:

```
let user = { name: "Deepak", age: 21 };
console.log("User is:", user);
console.log(2 + 3);   // 5
```

Why is it used?

Developers use it to check whether the code is running correctly and to see the value of a variable at any point.

Interview Answer:

console.log() is used to print values in the console. It is the simplest way to debug JavaScript code and check what a variable holds.

Important Point:

- console.log() does not change the webpage, it only prints.
- Other useful ones are console.error(), console.warn() and console.table().

Q4. Why do we use console.log()?

Easy English:

We use it mainly for debugging.

- To check if a function is really being called.
- To see the value of a variable, API response or error.
- It is fast and does not need any extra tool.

Hinglish:

Hum console.log() isliye use karte hain kyunki debugging aasan ho jaati hai. Function chal raha hai ya nahi, API se kya data aaya, variable me kya value hai - sab console me dikh jaata hai.

Example:

```
async function getData() {
  const res = await fetch("/api/users");
  const data = await res.json();
  console.log("API data:", data);   // check the response
}
```

Why is it used?

It is used because finding a bug is much easier when we can see the actual values at each step.

Interview Answer:

We use console.log() to debug. It shows the real value of variables and API responses, so we can quickly find where the code is going wrong.

Important Point:

- Remove or reduce console.log() in production code.
- Too many logs can slow down and confuse.

Q5. What is document.write()?

Easy English:

document.write() is an old JavaScript method that writes content directly into the HTML document.

- It writes text or HTML on the page while the page is loading.

Hinglish:

document.write() ek purana tarika hai jisse hum seedha HTML page par kuch likh sakte hain. Page load hote waqt yeh content page me daal deta hai.

Example:

```
document.write("<h1>Hello Deepak</h1>");
```

Why is it used?

It was used in old websites for quickly showing text before modern DOM methods existed.

Interview Answer:

document.write() writes content directly into the HTML page. It is an old method and is not used in modern applications.

Important Point:

- It is not recommended today.
- Modern code uses innerHTML, textContent or React rendering.

Q6. Why should we avoid document.write() in modern applications?

Easy English:

We avoid it because of these problems:

- If it runs after the page is fully loaded, it erases the whole existing page.
- It blocks HTML parsing and makes the page slower.
- It does not work well with React, Angular or any modern framework.
- It is unsafe if we write user input directly (security risk).

Hinglish:

document.write() isliye avoid karte hain kyunki agar page load hone ke baad yeh chale to poora page mit jata hai. Yeh page ko slow bhi karta hai aur React jaise modern frameworks ke saath kaam nahi karta. Iski jagah DOM methods use karte hain.

Example:

```
// Avoid this
document.write("Hello");

// Prefer this
document.getElementById("box").textContent = "Hello";
```

Why is it used?

Modern apps use safe and controlled DOM updates, so document.write() is replaced by textContent, innerHTML or framework rendering.

Interview Answer:

We avoid document.write() because it can overwrite the entire page after loading, slows down rendering, and does not fit modern frameworks. We use DOM methods or React instead.

Important Point:

- It overwrites the document if called late.

- textContent is safer than innerHTML for user data.

Q7. What is var?

Easy English:

var is the oldest way to declare a variable in JavaScript.

- It is function scoped, not block scoped.
- It can be re-declared and updated.
- It is hoisted and gets the value undefined before assignment.

Hinglish:

var JavaScript me variable banane ka sabse purana tarika hai. Yeh function scope follow karta hai, block scope nahi. Isko dobara declare bhi kar sakte hain, isliye galti hone ke chance zyada rehte hain.

Example:

```
var x = 10;
var x = 20;        // allowed, no error
if (true) { var y = 5; }
console.log(y);    // 5  (leaks out of the block)
```

Why is it used?

It was used before ES6. Today we avoid it because let and const give safer scoping.

Interview Answer:

var is the old way of declaring variables. It is function scoped, can be re-declared, and is hoisted as undefined. In modern JavaScript we prefer let and const.

Important Point:

- var ignores block scope like if and for.
- var is hoisted with value undefined.

Q8. What is let?

Easy English:

let was introduced in ES6 to declare variables whose value can change.

- It is block scoped, so it lives only inside { }.
- It can be updated but cannot be re-declared in the same scope.

Hinglish:

let ES6 me aaya tha. Yeh block scope hota hai, matlab curly braces ke andar hi kaam karta hai. Value badal sakte hain, lekin same scope me dobara declare nahi kar sakte.

Example:

```
let count = 1;
count = 2;          // allowed
// let count = 3;   // Error: already declared
if (true) { let z = 9; }
// console.log(z);  // Error: z is not defined
```

Why is it used?

We use let when the value of a variable will change later, for example a counter or a loop variable.

Interview Answer:

let is a block scoped way to declare a variable whose value can change. It cannot be re-declared in the same scope, which makes the code safer than var.

Important Point:

- let is hoisted but stays in the Temporal Dead Zone until it is assigned.
- Use let only when reassignment is needed.

Q9. What is const?

Easy English:

const is used to declare a variable whose reference cannot be changed.

- It is block scoped like let.
- It must be given a value at the time of declaration.
- For objects and arrays, the contents can still be changed, only re-assignment is blocked.

Hinglish:

const se aisa variable banate hain jiski value dobara assign nahi ki ja sakti. Yeh block scope hota hai aur declare karte waqt value dena zaroori hai. Object ya array ke andar ki value badal sakte hain, bas poora variable replace nahi kar sakte.

Example:

```
const pi = 3.14;
// pi = 3.15;            // Error

const user = { name: "Deepak" };
user.name = "Yadav";     // allowed (object content changed)
// user = {};            // Error (re-assignment)
```

Why is it used?

We use const by default so that values do not change by mistake. This makes code predictable and easy to read.

Interview Answer:

const declares a block scoped variable that cannot be re-assigned. Objects and arrays declared with const can still be modified internally, only the reference is fixed.

Important Point:

- const does not mean fully immutable.
- Best practice: use const first, use let only if the value must change.

Q10. Difference between var, let and const.

Easy English:

The three differ in scope, re-declaration, re-assignment and hoisting. The table below shows the main points.

Hinglish:

Teeno ka fark scope, dobara declare karne, value badalne aur hoisting me hai. Aaj ke code me var avoid karte hain, aur zyadatar const use karte hain.

Example:

```
var a = 1; var a = 2;     // ok
let b = 1; b = 2;           // ok, re-declare not ok
const c = 1;                // c = 2 -> Error
```

Why is it used?

We need to know the difference to avoid scope-related bugs and to write clean modern JavaScript.

Interview Answer:

var is function scoped, can be re-declared and re-assigned. let is block scoped and can be re-assigned but not re-declared. const is block scoped and cannot be re-assigned. In modern code we use const by default and let when the value changes.

Important Point:

- All three are hoisted, but let and const stay in the Temporal Dead Zone.
- var is the only one that leaks out of a block.

Point	var	let	const
Scope	Function	Block	Block
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No
Hoisting	Yes, as undefined	Yes, TDZ error	Yes, TDZ error
Use today	Avoid	When value changes	Default choice

Q11. What is a variable?

Easy English:

A variable is a named container that stores a value in memory.

- The value can be a number, string, boolean, array, object or function.
- We create it with var, let or const.

Hinglish:

Variable ek naam wala dabba hai jisme hum value store karte hain. Value number, string, array, object kuch bhi ho sakti hai. Variable banane ke liye var, let ya const use karte hain.

Example:

```
let city = "Gorakhpur";
const age = 21;
console.log(city, age);
```

Why is it used?

Variables are used to store and reuse data instead of writing the same value again and again.

Interview Answer:

A variable is a named memory container that stores a value. In JavaScript we declare it using let, const or var, and the value can be of any data type.

Important Point:

- JavaScript is dynamically typed, so a variable can hold any type.
- Variable names are case sensitive.

Q12. What is a data type?

Easy English:

A data type tells what kind of value a variable holds. JavaScript has two groups:

- Primitive: string, number, boolean, null, undefined, symbol, bigint.
- Non-primitive (reference): object, array, function.
- Primitives are copied by value, objects are copied by reference.

Hinglish:

Data type batata hai ki variable me kis tarah ki value hai. Primitive types hote hain jaise string, number, boolean, null, undefined. Aur non-primitive hote hain jaise object, array, function. Primitive value copy hoti hai, object reference se copy hota hai.

Example:

```
let s = "hello";        // string
let n = 25;               // number
let ok = true;            // boolean
let arr = [1, 2, 3];      // object (array)
let obj = { id: 1 };      // object
```

Why is it used?

Knowing data types helps us avoid wrong comparisons, wrong calculations and unexpected bugs.

Interview Answer:

A data type defines the kind of value stored in a variable. JavaScript has primitive types like string, number, boolean, null and undefined, and reference types like object, array and function.

Important Point:

- typeof null returns "object", which is a famous JavaScript bug.
- Arrays are technically objects.

Q13. What is typeof?

Easy English:

typeof is an operator that returns the data type of a value as a string.

- It is used to check the type before using a value.

Hinglish:

typeof ek operator hai jo batata hai ki value ka type kya hai, aur wo type string ke roop me return karta hai. Value use karne se pehle check karne ke kaam aata hai.

Example:

```
console.log(typeof "hi");      // string
console.log(typeof 10);         // number
console.log(typeof true);       // boolean
console.log(typeof undefined);  // undefined
console.log(typeof null);       // object  (known bug)
console.log(typeof [1,2]);      // object
console.log(typeof function(){})// function
```

Why is it used?

It is used for type checking, for example before calling a function or doing a calculation on API data.

Interview Answer:

typeof is an operator that returns the type of a value as a string. It is commonly used for type checking, but remember that typeof null returns "object".

Important Point:

- Use Array.isArray() to check arrays, not typeof.
- typeof on an undeclared variable returns "undefined" without error.

Q14. What is null?

Easy English:

null is a special value that means "empty on purpose" or "no value".

- The developer assigns it intentionally.
- Its type is object because of an old JavaScript bug.

Hinglish:

null ka matlab hai jaan-boojh kar khaali value rakhna. Developer khud set karta hai ki yahan abhi koi value nahi hai. typeof null "object" deta hai, jo JavaScript ka purana bug hai.

Example:

```
let user = null;       // we know it is empty for now
console.log(user);       // null
console.log(typeof user) // object
```

Why is it used?

It is used to clear a value or to show that data is not available yet, for example before an API response arrives.

Interview Answer:

null is an intentional empty value assigned by the developer. It means the variable exists but currently holds no value.

Important Point:

- typeof null is "object".
- null == undefined is true, but null === undefined is false.

Q15. What is undefined?

Easy English:

undefined means a variable has been declared but no value has been given to it.

- JavaScript sets it automatically.
- A function that returns nothing also returns undefined.
- A missing object property gives undefined.

Hinglish:

undefined ka matlab hai variable bana hai lekin usme value nahi daali gayi. Ye JavaScript khud set karta hai. Function agar kuch return na kare to bhi undefined milta hai.

Example:

```
let x;
console.log(x);            // undefined

const obj = { a: 1 };
console.log(obj.b);        // undefined

function f() {}
console.log(f());          // undefined
```

Why is it used?

It helps us know that a value was never set, which is useful while checking API data or function results.

Interview Answer:

undefined means a variable is declared but has no value assigned. JavaScript sets it automatically, unlike null which the developer sets manually.

Important Point:

- Never assign undefined manually, use null instead.
- Accessing a missing property returns undefined, not an error.

Q16. Difference between null and undefined.

Easy English:

Both mean "no value", but the reason is different.

- undefined: JavaScript itself says no value is assigned.
- null: the developer intentionally says the value is empty.

Hinglish:

Dono ka matlab "koi value nahi" hai, lekin fark yeh hai ki undefined JavaScript khud deta hai, aur null hum khud set karte hain.

Example:

```
let a;              // undefined (by JavaScript)
let b = null;         // null (by developer)

console.log(a == b);   // true  (loose)
console.log(a === b);  // false (types differ)
```

Why is it used?

Knowing the difference helps us write correct checks in API responses and form data.

Interview Answer:

undefined means the value was never assigned and JavaScript sets it automatically. null is an empty value set intentionally by the developer. null == undefined is true, but null === undefined is false.

Important Point:

- typeof undefined is "undefined", typeof null is "object".
- Use optional chaining and nullish coalescing to handle both safely.

Point	null	undefined
Meaning	Intentionally empty	Value not assigned
Set by	Developer	JavaScript
typeof	object	undefined
Use	Clear a value	Default for new variables

Q17. What is NaN?

Easy English:

NaN means "Not a Number". It is a special number value that appears when a maths operation fails.

- Example: multiplying a text with a number.
- NaN is not equal to anything, not even to itself.

Hinglish:

NaN ka matlab hai "Not a Number". Jab koi maths operation fail ho jaata hai, jaise string ko number se multiply karna, tab NaN aata hai. Interesting baat - NaN khud ke barabar bhi nahi hota.

Example:

```
console.log("abc" * 2);       // NaN
console.log(typeof NaN);        // number
console.log(NaN === NaN);       // false
console.log(Number.isNaN(NaN)); // true
```

Why is it used?

We check for NaN to avoid showing wrong values to the user when converting form input or API data to numbers.

Interview Answer:

NaN stands for Not a Number. It is returned when a numeric operation fails, for example "abc" * 2. Its type is number, and we check it using Number.isNaN().

Important Point:

- NaN === NaN is false.
- Use Number.isNaN() instead of the old global isNaN().

Q18. What is type coercion?

Easy English:

Type coercion means JavaScript automatically converts one data type into another.

- It happens with operators like + and ==.
- The + operator with a string converts numbers into strings.
- It can create surprising results, so we should be careful.

Hinglish:

Type coercion ka matlab hai JavaScript khud ek type ko doosre type me badal deti hai. Jaise "5" + 2 karne par string ban jaata hai "52". Isi wajah se == ki jagah === use karna better hai.

Example:

```
console.log("5" + 2);    // "52"  (number -> string)
console.log("5" - 2);    // 3     (string -> number)
console.log(1 == "1");   // true  (coercion)
console.log(1 === "1");  // false (no coercion)
```

Why is it used?

We need to understand coercion because it explains many strange bugs, especially when data comes from forms or APIs as strings.

Interview Answer:

Type coercion is JavaScript automatically converting one data type into another, for example a number into a string during comparison or concatenation. To avoid it, we use === and convert values explicitly with Number() or String().

Important Point:

- Implicit coercion is done by JavaScript, explicit conversion is done by us.
- == does coercion, === does not.

Q19. What is strict equality ===?

Easy English:

=== is the strict equality operator.

- It compares both value and data type.
- It does not convert types, so the result is predictable.

Hinglish:

=== strict equality operator hai. Yeh value aur data type dono compare karta hai aur type ko convert nahi karta. Isliye result hamesha clear aur safe hota hai.

Example:

```
console.log(5 === 5);     // true
console.log(5 === "5");   // false (type differs)
console.log(0 === false); // false
```

Why is it used?

We use === to avoid unexpected results caused by type coercion, which makes the code safer and easier to debug.

Interview Answer:

=== is strict equality. It compares both value and type without converting anything, so 5 === "5" is false. It is the recommended way to compare in JavaScript.

Important Point:

- !== is the strict not-equal operator.
- Always prefer === over == in interviews and real code.

Q20. Difference between == and ===.

Easy English:

Both compare two values, but the way is different.

- == compares only value and converts types first (loose equality).
- === compares value and type together (strict equality).

Hinglish:

== sirf value dekhta hai aur zarurat padne par type badal deta hai. === value aur type dono check karta hai. Isliye hamesha === use karna chahiye.

Example:

```
console.log(5 == "5");     // true
console.log(5 === "5");    // false
console.log(null == undefined);   // true
console.log(null === undefined);  // false
```

Why is it used?

Developers prefer === because it prevents hidden bugs caused by automatic type conversion.

Interview Answer:

== is loose equality and compares values after type conversion. === is strict equality and compares both value and type without conversion. We always prefer === in real projects.

Important Point:

- 0 == false is true, but 0 === false is false.
- null == undefined is true, only with ==.

Point	== (loose)	=== (strict)
Compares	Value only	Value and type
Type conversion	Yes	No
5 == "5"	true	false
Recommended	No	Yes

Functions (Q21 - Q30)

Q21. What is a function?

Easy English:

A function is a block of code that performs one specific task.

- We write it once and call it many times.
- It can take input (parameters) and return output.

Hinglish:

Function code ka ek block hota hai jo ek specific kaam karta hai. Ek baar likh kar baar baar call kar sakte hain. Function input le sakta hai aur result return kar sakta hai.

Example:

```
function add(a, b) {
  return a + b;
}
console.log(add(2, 3));   // 5
```

Why is it used?

Functions are used to reuse code, reduce repetition and break a big problem into small readable parts.

Interview Answer:

A function is a reusable block of code that performs a specific task. It can accept parameters and return a value, which helps keep code clean and reusable.

Important Point:

- Parameters are in the definition, arguments are the real values passed.
- In JavaScript, functions are first-class citizens.

Q22. Why do we use functions?

Easy English:

We use functions because:

- They avoid writing the same code again and again (DRY principle).
- They make code readable and organised.
- They are easy to test and debug.
- They allow us to pass behaviour to other functions (callbacks).

Hinglish:

Function isliye use karte hain kyunki same code baar baar likhna nahi padta, code saaf aur samajhne laayak rehta hai, aur testing-debugging aasan ho jaati hai. Ek jagah change karo, sab jagah change ho jaata hai.

Example:

```
function calcTotal(price, qty) {
  return price * qty;
}
console.log(calcTotal(100, 2));   // 200
console.log(calcTotal(50, 4));    // 200
```

Why is it used?

They are used to keep logic in one place so that changing it once updates the whole application.

Interview Answer:

We use functions for code reusability and clean structure. They let us write logic once, call it many times, and make the code easier to test and maintain.

Important Point:

- Functions support the DRY principle: Do not Repeat Yourself.
- Small functions are easier to debug than big ones.

Q23. What is a function declaration?

Easy English:

A function declaration creates a named function using the function keyword.

- It is fully hoisted, so it can be called before it is written in the code.

Hinglish:

Function declaration matlab function keyword ke saath naam wala function banana. Yeh poori tarah hoist hota hai, isliye define karne se pehle bhi call kar sakte hain.

Example:

```
greet();   // works because of hoisting

function greet() {
  console.log("Hello Deepak");
}
```

Why is it used?

It is used for normal reusable functions where hoisting is helpful and the name shows clearly in the stack trace.

Interview Answer:

A function declaration is a named function created with the function keyword. It is hoisted completely, so we can call it before its definition in the code.

Important Point:

- Only declarations are fully hoisted, not expressions.
- The function name appears in error stack traces, which helps debugging.

Q24. What is a function expression?

Easy English:

A function expression means storing a function inside a variable.

- It can be named or anonymous.
- It is not hoisted like a declaration, so it must be defined before use.

Hinglish:

Function expression matlab function ko variable me store karna. Yeh hoist nahi hota jaise declaration hota hai, isliye pehle likhna zaroori hai, tabhi call kar sakte hain.

Example:

```
// greet();   // Error: cannot access before initialization

const greet = function () {
  console.log("Hello");
```

```
};
greet();   // works
```

Why is it used?

It is used when we want to pass a function as a value, for example as a callback or inside an object.

Interview Answer:

A function expression is a function stored in a variable. Unlike a function declaration, it is not hoisted, so we must define it before calling it.

Important Point:

- Function expressions are useful as callbacks.
- Arrow functions are a shorter form of function expression.

Q25. What is an arrow function?

Easy English:

An arrow function is a short way to write a function, introduced in ES6.

- It uses the => symbol.
- It does not have its own this, arguments or prototype.
- A single expression body returns the value automatically.

Hinglish:

Arrow function ES6 me aaya chhota syntax hai jisme => use hota hai. Iska apna this nahi hota, yeh parent ka this leta hai. Ek line wale function me return likhna bhi nahi padta.

Example:

```
const add = (a, b) => a + b;     // implicit return
const square = n => n * n;
const hi = () => { console.log("hi"); };

console.log(add(2, 3));   // 5
```

Why is it used?

Arrow functions are used for short callbacks, array methods like map and filter, and inside React components where this behaviour matters.

Interview Answer:

An arrow function is a shorter ES6 syntax for writing functions. It has no own this, so it takes this from its parent scope, which makes it very useful in callbacks and React.

Important Point:

- Arrow functions cannot be used as constructors.
- They do not have the arguments object.

Point	Normal function	Arrow function
Syntax	function name() {}	const name = () => {}
this	Own this	Takes parent this
Hoisting	Declaration is hoisted	Not hoisted
Constructor (new)	Yes	No
arguments object	Yes	No

Q26. Why do we use arrow functions?

Easy English:

We use arrow functions because:

- The syntax is short and clean.

- They automatically take this from the surrounding scope, which removes many this related bugs.
- They are perfect as callbacks in map, filter, setTimeout and event handlers.

Hinglish:

Arrow function isliye use karte hain kyunki syntax chhota hota hai aur this ka problem khatam ho jaata hai. Yeh parent ka this leta hai, isliye callbacks aur React ke andar likhna aasan ho jaata hai.

Example:

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);
console.log(doubled);    // [2, 4, 6]

setTimeout(() => console.log("done"), 1000);
```

Why is it used?

They are used to write cleaner functional-style code, especially with array methods and asynchronous callbacks.

Interview Answer:

We use arrow functions because they give short syntax and do not create their own this. This makes callbacks and React code cleaner and avoids common this binding problems.

Important Point:

- Do not use arrow functions as object methods when you need this of the object.
- Use normal functions for constructors and prototype methods.

Q27. What is a callback function?

Easy English:

A callback is a function passed as an argument to another function, to be called later.

- It is used for asynchronous work like reading a file or calling an API.
- Array methods like map, filter and forEach also take callbacks.

Hinglish:

Callback wo function hai jo doosre function ko argument ki tarah diya jaata hai aur baad me call hota hai. Ye async kaam jaise API call ya file read karne me use hota hai, aur map/filter me bhi.

Example:

```
function getData(callback) {
  setTimeout(() => {
    callback("Data received");
  }, 1000);
}

getData(msg => console.log(msg));   // Data received
```

Why is it used?

Callbacks are used so that some code runs only after another task finishes, without blocking the rest of the program.

Interview Answer:

A callback function is a function passed as an argument to another function and executed later. It is mainly used for asynchronous operations so that code runs after a task completes.

Important Point:

- Too many nested callbacks create "callback hell".
- Promises and async/await were introduced to solve callback hell.

Q28. What is a higher-order function?

Easy English:

A higher-order function is a function that either takes a function as an argument or returns a function.

- Examples: map, filter, reduce, forEach, setTimeout.

Hinglish:

Higher-order function wo hota hai jo ya to function ko argument me leta hai, ya function return karta hai. map, filter, reduce iske common examples hain.

Example:

```
const nums = [1, 2, 3, 4];
const even = nums.filter(n => n % 2 === 0);
console.log(even);        // [2, 4]

function multiplier(x) {
  return n => n * x;      // returns a function
}
console.log(multiplier(3)(5));   // 15
```

Why is it used?

They are used to write reusable and clean functional code instead of long loops.

Interview Answer:

A higher-order function takes a function as an argument or returns a function. Common examples are map, filter and reduce, which make array operations clean and readable.

Important Point:

- This is possible because functions are first-class citizens in JavaScript.
- map returns a new array, forEach returns undefined.

Q29. What is a pure function?

Easy English:

A pure function has two rules:

- For the same input, it always gives the same output.
- It does not change anything outside itself (no side effects).
- It does not modify global variables, the DOM or the original array.

Hinglish:

Pure function wo hai jo same input par hamesha same output de, aur bahar ki koi cheez na badle. Matlab global variable, DOM ya original array ko touch na kare. Isse code predictable ban jaata hai.

Example:

```
// Pure
function add(a, b) { return a + b; }

// Impure (changes outside value)
let total = 0;
function addToTotal(n) { total += n; }
```

Why is it used?

Pure functions are used because they are predictable, easy to test and safe to reuse. React state updates also follow this idea.

Interview Answer:

A pure function always returns the same output for the same input and has no side effects. It makes code predictable and easy to test, and React follows this concept for state updates.

Important Point:

- React reducers should be pure functions.
- Methods like map and filter are pure, push and splice are not.

Q30. What is recursion?

Easy English:

Recursion is when a function calls itself to solve a smaller part of the same problem.

- Every recursion needs a base condition to stop.
- Without a base case it runs forever and gives a stack overflow error.

Hinglish:

Recursion ka matlab hai function ka khud ko call karna. Har recursion me ek base condition honi chahiye jahan rukna hai, warna stack overflow error aa jaayega.

Example:

```
function factorial(n) {
  if (n <= 1) return 1;        // base condition
  return n * factorial(n - 1); // recursive call
}
console.log(factorial(5));     // 120
```

Why is it used?

It is used for problems with a repeating structure, such as factorial, Fibonacci, tree traversal and nested folder or comment structures.

Interview Answer:

Recursion is when a function calls itself with a smaller input until it reaches a base condition. It is used for problems like factorial, tree traversal and nested data.

Important Point:

- A missing base case causes "Maximum call stack size exceeded".
- Loops are usually faster, recursion is cleaner for tree-like data.

Advanced JavaScript (Q31 - Q50)

Q31. What is scope?

Easy English:

Scope decides where a variable can be accessed in the code. JavaScript has:

- Global scope: available everywhere.
- Function scope: available only inside that function.
- Block scope: available only inside { } for let and const.

Hinglish:

Scope batata hai ki variable kahan kahan use ho sakta hai. Global scope me variable har jagah milta hai, function scope me sirf us function ke andar, aur block scope me sirf curly braces ke andar (let/const ke liye).

Example:

```
let g = "global";

function test() {
  let f = "function";
  if (true) {
    let b = "block";
    console.log(g, f, b);   // all work
  }
  // console.log(b);        // Error
}
test();
```

Why is it used?

Scope is used to protect variables, avoid name conflicts and keep memory clean.

Interview Answer:

Scope defines where a variable is accessible. JavaScript has global scope, function scope and block scope. let and const are block scoped while var is function scoped.

Important Point:

- Inner scope can read outer variables, but not the opposite.
- Too many global variables cause conflicts.

Q32. What is lexical scope?

Easy English:

Lexical scope means an inner function can access variables of its outer function, based on where the code is written.

- The scope is decided at the time of writing the code, not at runtime.

Hinglish:

Lexical scope ka matlab hai andar wala function bahar wale function ki value use kar sakta hai, kyunki wo uske andar likha gaya hai. Ye position se decide hota hai, chalane ke time par nahi.

Example:

```
function outer() {
  const name = "Deepak";
  function inner() {
    console.log(name);   // can read outer variable
  }
  inner();
}
outer();
```

Why is it used?

It is used to understand how nested functions read data and it is the base concept behind closures.

Interview Answer:

Lexical scope means a function can access variables from its outer scope based on where it is written in the code. This is the foundation of closures in JavaScript.

Important Point:

- Lexical scope is also called static scope.
- Closures exist because of lexical scope.

Q33. What is closure?

Easy English:

A closure is a function that remembers the variables of its outer function even after the outer function has finished.

- The inner function keeps access to the outer variables.
- It is used to create private data and to keep values between calls.

Hinglish:

Closure ka matlab hai andar wala function bahar wale function ki value yaad rakhta hai, chahe bahar wala function khatam ho chuka ho. Isse private variable aur counter jaise cheezein banti hain.

Example:

```
function counter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const inc = counter();
console.log(inc());   // 1
```

```
console.log(inc());   // 2  (count is remembered)
```

Why is it used?

Closures are used for data privacy, counters, once-only functions, and in React hooks where state is remembered between renders.

Interview Answer:

A closure is a function that remembers variables from its outer scope even after the outer function has returned. It is used for data privacy and to keep values between function calls.

Important Point:

- Closures are created every time a function is created.
- They can cause memory leaks if references are kept unnecessarily.

Q34. What is hoisting?

Easy English:

Hoisting means JavaScript moves declarations to the top of their scope before running the code.

- var is hoisted with the value undefined.
- let and const are hoisted but stay in the Temporal Dead Zone, so using them early gives an error.
- Function declarations are hoisted completely.

Hinglish:

Hoisting ka matlab hai JavaScript declarations ko upar utha leti hai code chalne se pehle. var hoist hoke undefined ban jaata hai, let/const hoist to hote hain lekin unko pehle use karne par error aata hai. Function declaration poora hoist hota hai.

Example:

```
console.log(a);   // undefined
var a = 5;

// console.log(b); // ReferenceError (TDZ)
let b = 10;

sayHi();            // works
function sayHi() { console.log("hi"); }
```

Why is it used?

Understanding hoisting explains why some variables are undefined and why some give an error before declaration.

Interview Answer:

Hoisting is JavaScript moving declarations to the top of their scope. var becomes undefined, let and const stay in the Temporal Dead Zone, and function declarations are fully hoisted.

Important Point:

- Only declarations are hoisted, not the assigned values.
- Function expressions and arrow functions are not hoisted like declarations.

Q35. What is execution context?

Easy English:

Execution context is the environment in which JavaScript code runs.

- Global execution context is created first.
- A new function execution context is created for every function call.
- Each context has two phases: memory creation (hoisting) and code execution.

Hinglish:

Execution context wo environment hai jisme JavaScript code chalta hai. Sabse pehle global context banta hai, phir har function call par naya context banta hai. Har context ke do phase hote hain - memory phase aur execution phase.

Example:

```
let x = 10;            // global context

function show() {       // new context on call
  let y = 20;
  console.log(x + y);
}
show();                 // 30
```

Why is it used?

It is used to understand how variables, this, and hoisting really work inside JavaScript.

Interview Answer:

Execution context is the environment where JavaScript code is executed. A global context is created first, and a new context is created for each function call. Each has a memory phase and an execution phase.

Important Point:

- Each context stores variable environment, scope chain and this.
- Contexts are managed using the call stack.

Q36. What is call stack?

Easy English:

The call stack is a structure that keeps track of which function is running.

- It works on LIFO (Last In First Out).
- When a function is called it is pushed, when it returns it is popped.
- If it becomes too full, we get a stack overflow error.

Hinglish:

Call stack ek list hai jo yaad rakhti hai kaunsa function abhi chal raha hai. Function call hone par stack me add hota hai aur khatam hone par hat jaata hai. Ye LIFO order follow karta hai.

Example:

```
function one() { two(); }
function two() { console.log("two"); }
one();
// Stack: one -> two -> two removed -> one removed
```

Why is it used?

It is used by JavaScript to manage function execution order and to show error stack traces.

Interview Answer:

The call stack tracks which function is currently running. Functions are pushed when called and popped when they return, following Last In First Out order. Infinite recursion causes a stack overflow.

Important Point:

- JavaScript has only one call stack because it is single-threaded.
- Heavy synchronous code blocks the stack and freezes the page.

Q37. What is event loop?

Easy English:

The event loop is the mechanism that lets single-threaded JavaScript handle asynchronous work.

- It keeps checking if the call stack is empty.
- When the stack is empty, it moves tasks from the queues into the stack.
- Microtasks (Promises) are given priority over callback queue tasks (setTimeout).

Hinglish:

Event loop ki wajah se single-threaded JavaScript async kaam kar paati hai. Ye baar baar dekhta hai ki call stack khaali hai ya nahi. Khaali hote hi queue se kaam uthakar stack me daal deta hai. Promise wale microtask pehle chalte hain, setTimeout baad me.

Example:

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1 4 3 2
```

Why is it used?

It is used so that timers, API calls and file operations do not block the main thread.

Interview Answer:

The event loop checks whether the call stack is empty and then pushes pending callbacks from the microtask and callback queues into the stack. This is how single-threaded JavaScript handles asynchronous tasks.

Important Point:

- Microtask queue has higher priority than callback (macrotask) queue.
- The event loop is the most asked async question in interviews.

Q38. What is callback queue?

Easy English:

The callback queue (also called task queue or macrotask queue) holds callbacks that are ready to run.

- Callbacks of setTimeout, setInterval and DOM events wait here.
- The event loop moves them to the call stack only when the stack is empty.

Hinglish:

Callback queue me wo callbacks line lagate hain jo chalne ke liye taiyaar hain, jaise setTimeout aur click event ke callbacks. Event loop unko tabhi stack me bhejta hai jab stack khaali ho.

Example:

```
setTimeout(() => console.log("timer done"), 0);
console.log("sync code");
// Output: sync code  then  timer done
```

Why is it used?

It is used to store async callbacks so that the main thread keeps running without waiting.

Interview Answer:

The callback queue stores callbacks from setTimeout, setInterval and DOM events. The event loop pushes them to the call stack only after all synchronous code is finished.

Important Point:

- setTimeout with 0 ms still waits for the stack to be empty.
- Microtasks always run before the callback queue.

Q39. What is microtask queue?

Easy English:

The microtask queue holds high priority callbacks.

- Promise .then(), .catch(), .finally() and queueMicrotask go here.
- All microtasks run before any callback queue task.

Hinglish:

Microtask queue me zyada priority wale kaam aate hain, jaise Promise ke .then aur .catch. Event loop pehle poori microtask queue khaali karta hai, uske baad setTimeout jaise task chalte hain.

Example:

```
setTimeout(() => console.log("macro"), 0);
Promise.resolve().then(() => console.log("micro"));
// Output: micro  then  macro
```

Why is it used?

It is used so that promise results are handled as early as possible, right after the current code finishes.

Interview Answer:

The microtask queue holds Promise callbacks and has higher priority than the callback queue. The event loop empties all microtasks before running any setTimeout callback.

Important Point:

- async/await internally uses the microtask queue.
- Endless microtasks can starve the macrotask queue.

Q40. What is Promise?

Easy English:

A Promise is an object that represents the result of an asynchronous operation.

- It has three states: pending, fulfilled and rejected.
- We handle success with .then() and error with .catch().
- .finally() runs in both cases.

Hinglish:

Promise ek object hai jo async kaam ka result represent karta hai. Iske teen state hote hain - pending, fulfilled aur rejected. Success .then me milti hai aur error .catch me.

Example:

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Success!"), 1000);
});

p.then(res => console.log(res))
 .catch(err => console.log(err))
 .finally(() => console.log("done"));
```

Why is it used?

It is used to handle asynchronous tasks like API calls in a clean way instead of deeply nested callbacks.

Interview Answer:

A Promise is an object that represents a future value of an asynchronous operation. It can be pending, fulfilled or rejected, and we handle it with .then and .catch.

Important Point:

- A promise can settle only once.
- Promise.all, Promise.race and Promise.allSettled handle multiple promises.

Q41. Why do we use Promise?

Easy English:

We use Promises because:

- They remove callback hell and make code readable.
- They give clean error handling with .catch().
- They can be chained one after another.
- They work with async/await.

Hinglish:

Promise isliye use karte hain kyunki callback hell khatam ho jaata hai aur code padhne me aasan rehta hai. Error handling .catch se simple ho jaati hai aur async/await ke saath aur clean code milta hai.

Example:

```
fetch("https://api.example.com/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log("Error:", err));
```

Why is it used?

They are used for API calls, database queries and any task whose result comes later.

Interview Answer:

We use Promises to handle asynchronous operations in a clean and readable way. They avoid callback hell, support chaining, and give centralised error handling with .catch.

Important Point:

- Promise.all runs multiple promises in parallel.
- Always attach a .catch, otherwise you get an unhandled rejection.

Q42. What is async/await?

Easy English:

async/await is modern syntax built on top of Promises.

- The async keyword makes a function return a Promise.
- await pauses the function until the Promise settles.
- Errors are handled using try...catch.

Hinglish:

async/await Promise ka hi naya aur aasan syntax hai. async lagane se function Promise return karta hai, aur await likhne par function ruk jaata hai jab tak result nahi aata. Error try...catch se handle karte hain.

Example:

```
async function getUser() {
  try {
    const res = await fetch("/api/user");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error:", err.message);
  }
}
getUser();
```

Why is it used?

It is used because asynchronous code looks like normal synchronous code, which is much easier to read and debug.

Interview Answer:

async/await is syntax built on Promises. async makes a function return a Promise and await waits for the result, so asynchronous code looks like synchronous code and is easier to read.

Important Point:

- await works only inside an async function (or top level in ES modules).
- await does not block the whole program, only that function.

Q43. Difference between Promise and async/await.

Easy English:

Both handle asynchronous code, and async/await internally uses Promises.

- Promise uses .then() and .catch() chaining.

- async/await uses await and try...catch, which looks cleaner.

Hinglish:

Dono async kaam ke liye hain, aur async/await andar se Promise hi use karta hai. Promise me .then chain hoti hai, jabki async/await me code seedha upar se neeche padha jaata hai, isliye samajhna aasan hota hai.

Example:

```
// Promise style
fetch(url).then(r => r.json()).then(d => console.log(d));

// async/await style
async function load() {
  const r = await fetch(url);
  const d = await r.json();
  console.log(d);
}
```

Why is it used?

async/await is used in modern projects because the code is more readable, especially when there are many steps.

Interview Answer:

Promise and async/await both handle asynchronous tasks, and async/await is syntactic sugar over Promises. Promise uses .then and .catch chains, while async/await uses await with try...catch and reads like synchronous code.

Important Point:

- Use Promise.all with await to run tasks in parallel.
- Writing await one after another runs tasks one by one, which can be slow.

Point	Promise	async/await
Syntax	.then() / .catch()	await + try...catch
Readability	Chained, can get long	Looks synchronous, cleaner
Error handling	.catch()	try...catch
Base	Core feature	Built on Promises

Q44. What is try...catch?

Easy English:

try...catch is used to handle errors without crashing the program.

- Code that may fail goes in try.
- If an error happens, control jumps to catch.
- finally runs in both success and failure.

Hinglish:

try...catch error handle karne ke liye hai. Jo code fail ho sakta hai wo try me likhte hain, aur error aane par catch chalta hai. finally dono situation me chalta hai.

Example:

```
try {
  const data = JSON.parse("invalid json");
} catch (err) {
  console.log("Error:", err.message);
} finally {
  console.log("Always runs");
}
```

Why is it used?

It is used to show a proper message to the user instead of letting the application break.

Interview Answer:

try...catch handles runtime errors. The risky code goes inside try, and if it throws, the catch block handles it. finally always runs, and we use it with async/await for API errors.

Important Point:

- try...catch does not catch errors inside plain async callbacks automatically.
- In Express we use error-handling middleware along with try...catch.

Q45. What is destructuring?

Easy English:

Destructuring is a short way to take values out of an array or object and put them into variables.

- Object destructuring matches by key name.
- Array destructuring matches by position.
- We can also set default values.

Hinglish:

Destructuring ka matlab hai array ya object se value nikal kar seedha variable me daal dena. Object me key ke naam se aur array me position se value milti hai. Default value bhi de sakte hain.

Example:

```
const user = { name: "Deepak", city: "Gorakhpur" };
const { name, city, age = 21 } = user;
console.log(name, city, age);

const [a, b] = [10, 20];
console.log(a, b);      // 10 20
```

Why is it used?

It is used to write shorter code, especially with API responses, React props and Express req objects.

Interview Answer:

Destructuring lets us extract values from objects and arrays into variables in one line. It is widely used for React props, API responses and req.body in Express.

Important Point:

- Object destructuring depends on key names, array on order.
- We can rename while destructuring: const { name: userName } = user.

Q46. What is spread operator?

Easy English:

The spread operator (...) expands an array or object into individual items.

- It is used to copy arrays and objects.
- It is used to merge two arrays or objects.
- It creates a shallow copy, not a deep copy.

Hinglish:

Spread operator (...) array ya object ko tod kar alag alag items me bana deta hai. Isse copy banti hai aur do arrays/objects merge ho jaate hain. Dhyaan rahe, ye shallow copy hai.

Example:

```
const a = [1, 2];
const b = [...a, 3];              // [1, 2, 3]

const user = { name: "Deepak" };
const newUser = { ...user, age: 21 };
console.log(newUser);             // { name: 'Deepak', age: 21 }
```

Why is it used?

It is used to update state in React without changing the original data, and to merge configuration objects.

Interview Answer:

The spread operator expands an array or object into its elements. It is used to copy and merge arrays or objects, and it is very common in React state updates.

Important Point:

- Spread creates a shallow copy, nested objects still share the reference.
- Same ... symbol is used for rest, but the position decides the meaning.

Q47. What is rest operator?

Easy English:

The rest operator (...) collects multiple values into one array or object.

- In a function it collects extra arguments.
- In destructuring it collects the remaining properties.
- It must be the last parameter.

Hinglish:

Rest operator (...) bahut saari values ko ek array ya object me jama karta hai. Function me extra arguments collect karta hai aur destructuring me bachi hui properties le leta hai. Ye hamesha last me aata hai.

Example:

```
function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}
console.log(sum(1, 2, 3));    // 6

const { name, ...rest } = { name: "D", age: 21, city: "GKP" };
console.log(rest);            // { age: 21, city: 'GKP' }
```

Why is it used?

It is used when we do not know how many arguments will come, or when we want to separate some keys from an object.

Interview Answer:

The rest operator collects multiple values into a single array or object. Spread expands values while rest gathers them, even though both use three dots.

Important Point:

- Rest must be the last parameter in a function.
- Rest is often used to remove a field, for example hiding the password from a user object.

Q48. What is template literal?

Easy English:

A template literal is a string written with backticks ( ).

- We can insert variables using ${ }.
- We can write multi-line strings easily.

Hinglish:

Template literal backtick wali string hoti hai. Isme ${} ke andar variable daal sakte hain aur multi-line string bhi bina jod-tod ke likh sakte hain.

Example:

```
const name = "Deepak";
const age = 21;
console.log(`My name is ${name} and I am ${age} years old.`);

const html = `
  <h1>${name}</h1>
```

```
`;
```

Why is it used?

It is used to build dynamic strings such as messages, URLs, HTML templates and MongoDB connection strings.

Interview Answer:

A template literal is a string written with backticks that supports variable interpolation using dollar-brace syntax and multi-line text. It replaces messy string concatenation.

Important Point:

- Template literals can hold expressions, not just variables.
- They are used for dynamic API URLs and email templates.

Q49. What is optional chaining (?.)?

Easy English:

Optional chaining lets us safely read a deeply nested property.

- If a value in the middle is null or undefined, it returns undefined instead of throwing an error.
- It works for properties, array items and function calls.

Hinglish:

Optional chaining (?.) se hum nested property safely padh sakte hain. Agar beech me koi value null ya undefined mili to error dene ki jagah undefined return ho jaata hai. Isse app crash nahi hota.

Example:

```
const user = { profile: { city: "Gorakhpur" } };

console.log(user.profile?.city);      // Gorakhpur
console.log(user.address?.city);      // undefined (no error)
// console.log(user.address.city);    // TypeError
```

Why is it used?

It is used with API responses where some fields may be missing, so the application does not crash.

Interview Answer:

Optional chaining lets us access nested properties safely. If any part is null or undefined, it returns undefined instead of throwing an error, which is very useful with API data.

Important Point:

- It only checks for null and undefined, not for empty strings or 0.
- It pairs well with nullish coalescing (??).

Q50. What is nullish coalescing (??)?

Easy English:

The ?? operator returns the right side value only when the left side is null or undefined.

- It is different from || which also treats 0, "" and false as empty.

Hinglish:

?? operator tabhi right side ki value deta hai jab left side null ya undefined ho. || se fark yeh hai ki || to 0 aur khaali string ko bhi falsy maan leta hai, jo kabhi kabhi galat result deta hai.

Example:

```
const count = 0;
console.log(count ?? 10);   // 0   (0 is a valid value)
console.log(count || 10);   // 10  (0 treated as falsy)

const name = null;
console.log(name ?? "Guest");   // Guest
```

Why is it used?

It is used to set default values correctly when 0 or an empty string is a valid value, for example price, quantity or rating.

Interview Answer:

The nullish coalescing operator returns the right side only if the left side is null or undefined. Unlike the OR operator, it

PART 2

NODE.JS

PART 2 - NODE.JS

Node.js (Q1 - Q30)

Q1. What is Node.js?

Easy English:

Node.js is a JavaScript runtime environment built on Google's V8 engine.

- It lets us run JavaScript outside the browser.
- It is mainly used to build backend servers and APIs.
- It is single-threaded and uses non-blocking asynchronous I/O.

Hinglish:

Node.js ek JavaScript runtime environment hai jo Google ke V8 engine par bana hai. Iski madad se JavaScript browser ke bahar bhi chalti hai, matlab server par. Isse backend aur API banate hain.

Example:

```
// server.js
const http = require("http");

http.createServer((req, res) => {
  res.end("Hello from Node.js");
}).listen(5000);
```

Why is it used?

It is used to build fast backend servers, REST APIs, real-time apps like chat, and command line tools.

Interview Answer:

Node.js is a JavaScript runtime environment based on the V8 engine. It allows us to run JavaScript outside the browser, mainly for backend development, APIs and real-time applications.

Important Point:

- Node.js is a runtime, not a framework and not a language.
- It uses non-blocking I/O, which makes it good for I/O heavy apps.

Q2. Why do we use Node.js?

Easy English:

We use Node.js because:

- One language (JavaScript) works on frontend and backend.
- It is fast because of the V8 engine.
- Non-blocking I/O handles many requests with few resources.
- npm gives access to a huge number of ready packages.

Hinglish:

Node.js isliye use karte hain kyunki frontend aur backend dono JavaScript me ban jaate hain. Ye V8 ki wajah se fast hai, non-blocking hone ke karan ek saath bahut saare requests handle kar leta hai, aur npm par lakhon packages milte hain.

Example:

```
const express = require("express");
const app = express();
app.get("/", (req, res) => res.send("API running"));
app.listen(5000, () => console.log("Server on port 5000"));
```

Why is it used?

It is used for REST APIs, chat apps, streaming and any application with many simultaneous connections.

Interview Answer:

We use Node.js because it lets us write backend code in JavaScript, it is fast due to the V8 engine, and its non-blocking I/O handles many concurrent requests efficiently.

Important Point:

- Node.js is not ideal for heavy CPU work like video processing.
- It is perfect for I/O heavy applications.

Q3. Is Node.js a programming language?

Easy English:

No. Node.js is not a programming language.

- It is a runtime environment that runs JavaScript code.
- JavaScript is the language, Node.js is the place where it runs on the server.

Hinglish:

Nahi, Node.js koi language nahi hai. Ye ek runtime environment hai jo JavaScript ko server par chalata hai. Language JavaScript hai, Node.js sirf usko chalane ka environment deta hai.

Example:

```
// This is JavaScript code
console.log("Hello");
// Run it with Node:  node app.js
```

Why is it used?

Understanding this is important because interviewers often test whether the candidate knows the difference between a language, a framework and a runtime.

Interview Answer:

No, Node.js is not a programming language. It is a JavaScript runtime environment built on the V8 engine that allows JavaScript to run on the server.

Important Point:

- JavaScript = language, Node.js = runtime, Express = framework.
- Node.js adds server features like fs, http and process which browsers do not have.

Q4. Is Node.js single-threaded?

Easy English:

Yes, Node.js runs our JavaScript code on a single main thread.

- It uses the event loop to handle asynchronous tasks.
- Heavy operations like file I/O are given to a thread pool (libuv) in the background.
- So it is single-threaded for JavaScript, but it uses multiple threads internally.

Hinglish:

Haan, Node.js ka JavaScript code ek hi main thread par chalta hai. Lekin async kaam event loop handle karta hai aur bhaari kaam libuv ke thread pool ko de diya jaata hai. Isliye single thread hote hue bhi ye bahut saare requests sambhal leta hai.

Example:

```
console.log("start");
setTimeout(() => console.log("async work"), 0);
console.log("end");
// start, end, async work
```

Why is it used?

This design is used so that the server does not wait for slow operations and can serve many users at the same time.

Interview Answer:

Yes, Node.js executes JavaScript on a single thread, but it handles concurrency using the event loop and a background thread pool provided by libuv, so it can serve many requests at once.

Important Point:

- Blocking code on the main thread freezes the whole server.
- For CPU heavy tasks we use worker threads or clustering.

Q5. What is V8 engine?

Easy English:

V8 is Google's open source JavaScript engine written in C++.

- It is used in Chrome and in Node.js.
- It converts JavaScript into machine code using JIT (Just In Time) compilation.
- This makes JavaScript execution very fast.

Hinglish:

V8 Google ka JavaScript engine hai jo C++ me bana hai. Ye Chrome aur Node.js dono me use hota hai. Ye JavaScript ko machine code me badal deta hai, isliye code tez chalta hai.

Example:

```
// Node.js runs this JS through the V8 engine
const sum = [1, 2, 3].reduce((a, b) => a + b, 0);
console.log(sum);   // 6
```

Why is it used?

It is used because interpreting JavaScript line by line would be slow, so V8 compiles it to machine code for speed.

Interview Answer:

V8 is Google's JavaScript engine written in C++. It compiles JavaScript directly into machine code, and Node.js uses it to run JavaScript fast on the server.

Important Point:

- V8 handles memory and garbage collection.
- Node.js = V8 engine + libuv + Node APIs.

Q6. What is npm?

Easy English:

npm stands for Node Package Manager.

- It is the default package manager that comes with Node.js.
- It is used to install, update and remove packages.
- It also runs project scripts.

Hinglish:

npm ka matlab hai Node Package Manager. Ye Node.js ke saath hi aata hai. Isse hum packages install, update aur delete karte hain, aur project ke scripts bhi chalate hain.

Example:

```
npm init -y              # create package.json
npm install express      # install a package
npm install -D nodemon   # dev dependency
npm uninstall express
```

Why is it used?

It is used so that we do not write everything from scratch. We simply install trusted libraries like express, mongoose and jsonwebtoken.

Interview Answer:

npm is the Node Package Manager that comes with Node.js. It is used to install and manage third-party packages and to run project scripts defined in package.json.

Important Point:

- npm is also the world's largest package registry.
- npx runs a package without installing it permanently.

Q7. What is package.json?

Easy English:

package.json is the configuration file of a Node.js project.

- It stores the project name, version and description.
- It lists dependencies and devDependencies.
- It contains scripts like start and dev.
- It is created with npm init.

Hinglish:

package.json project ki configuration file hai. Isme project ka naam, version, dependencies aur scripts likhe hote hain. Ye npm init se banti hai aur project ka pehchan patra maani jaati hai.

Example:

```
{
  "name": "mern-api",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": { "express": "^4.18.2" }
}
```

Why is it used?

It is used so that anyone can clone the project and install the exact same packages by running npm install.

Interview Answer:

package.json is the main configuration file of a Node project. It stores project details, dependencies and scripts, so the project can be installed and run anywhere with npm install.

Important Point:

- dependencies run in production, devDependencies are only for development.
- The caret symbol allows minor version updates.

Q8. What is node_modules?

Easy English:

node_modules is the folder where npm installs all packages.

- It contains our packages and all of their internal dependencies.
- It is very large, so we never push it to GitHub.
- It can be recreated anytime using npm install.

Hinglish:

node_modules wo folder hai jisme npm saare installed packages rakhta hai. Ye bahut bada hota hai, isliye GitHub par push nahi karte. Zarurat padne par npm install se dobara ban jaata hai.

Example:

```
# .gitignore
node_modules
.env
```

```
# recreate later
npm install
```

Why is it used?

It is used by Node.js to find and load the code of installed packages when we call require or import.

Interview Answer:

node_modules is the folder where all installed npm packages are stored. We add it to .gitignore because it is large and can be regenerated with npm install.

Important Point:

- Never edit files inside node_modules.
- package.json plus package-lock.json are enough to rebuild it.

Q9. What is package-lock.json?

Easy English:

package-lock.json records the exact version of every installed package.

- It locks versions so every developer gets the same setup.
- It is generated automatically by npm.
- It should be committed to Git.

Hinglish:

package-lock.json me har package ka exact version likha hota hai. Isse har developer ke system par same version install hota hai aur "mere system par chal raha tha" wali problem nahi aati. Ise Git me commit karna chahiye.

Example:

```
npm install          # updates package-lock.json
npm ci                 # installs exactly from the lock file
```

Why is it used?

It is used to make installations reproducible and to avoid bugs caused by a slightly different package version.

Interview Answer:

package-lock.json stores the exact versions of all installed dependencies. It makes installs consistent across machines, and unlike node_modules, it should be committed to Git.

Important Point:

- package.json has version ranges, package-lock has exact versions.
- npm ci is faster and used in CI/CD pipelines.

Q10. What is process?

Easy English:

process is a global object in Node.js that gives information about the running program.

- process.env gives environment variables.
- process.argv gives command line arguments.
- process.exit() stops the program.

Hinglish:

process Node.js ka global object hai jo chal rahe program ki jaankari deta hai. Isme environment variables, command line arguments aur program band karne ka option milta hai.

Example:

```
console.log(process.platform);   // linux / win32
console.log(process.version);    // v20.x.x
process.on("exit", () => console.log("Bye"));
```

Why is it used?

It is used to read configuration, handle shutdown and get runtime information about the server.

Interview Answer:

process is a global Node.js object that provides information and control over the current Node process, such as environment variables, command line arguments and exit handling.

Important Point:

- process is available without require.
- It does not exist in the browser, only in Node.js.

Q11. What is process.env?

Easy English:

process.env is an object that holds all environment variables.

- It is used to read secret values like database URL, JWT secret and port.
- Values are always read as strings.
- With dotenv we load them from a .env file.

Hinglish:

process.env ek object hai jisme environment variables hote hain. Isse hum secret cheezein padhte hain jaise database URL, JWT secret aur port. dotenv package .env file se ye values load kar deta hai.

Example:

```
require("dotenv").config();

const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI;
console.log(PORT);
```

Why is it used?

It is used to keep secrets out of the code and to use different settings for development and production.

Interview Answer:

process.env is an object containing environment variables. We use it with dotenv to read secrets like the Mongo URI, JWT secret and port, so sensitive values are never hard-coded.

Important Point:

- Every value in process.env is a string, so convert numbers with Number().
- Never commit real secret values to GitHub.

Q12. What is process.argv?

Easy English:

process.argv is an array of command line arguments.

- Index 0 is the node path, index 1 is the file path.
- Real user arguments start from index 2.

Hinglish:

process.argv ek array hai jisme command line se diye gaye arguments hote hain. Pehla item node ka path hota hai, doosra file ka path, aur asli arguments index 2 se shuru hote hain.

Example:

```
// node app.js Deepak 21
console.log(process.argv[2]);   // Deepak
console.log(process.argv[3]);   // 21
```

Why is it used?

It is used to build CLI tools and scripts that take input from the terminal, like seed or migration scripts.

Interview Answer:

process.argv is an array that contains the command line arguments passed to a Node script. The actual user arguments start from index 2.

Important Point:

- Arguments always come as strings.
- Libraries like yargs and commander parse them more easily.

Q13. What is EventEmitter?

Easy English:

EventEmitter is a Node.js class used to work with custom events.

- We use .on() to listen to an event.
- We use .emit() to fire an event.
- Many Node modules like http and streams are built on it.

Hinglish:

EventEmitter Node.js ki ek class hai jisse custom events banaye jaate hain. .on() se event sunte hain aur .emit() se event chalate hain. Node ke kai modules isi par bane hain.

Example:

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("order", (id) => {
  console.log("Order received:", id);
});

emitter.emit("order", 101);   // Order received: 101
```

Why is it used?

It is used to build event-driven applications where one action triggers other actions, for example sending an email after a user registers.

Interview Answer:

EventEmitter is a Node.js class that lets us create and handle custom events using on and emit. Node's event-driven architecture, including streams and HTTP, is built on it.

Important Point:

- Node.js follows an event-driven architecture.
- Too many listeners on one emitter causes a memory leak warning.

Q14. What is the Node.js Event Loop?

Easy English:

The event loop is the mechanism that lets Node.js perform non-blocking operations on a single thread.

- Async tasks are sent to libuv.
- When they are done, their callbacks are queued.
- The event loop pushes these callbacks to the call stack when it is empty.
- It runs in phases like timers, poll, check and close callbacks.

Hinglish:

Event loop ki wajah se Node.js single thread hote hue bhi non-blocking kaam kar paata hai. Async kaam libuv ko jaata hai, kaam poora hone par callback queue me aata hai, aur stack khaali hote hi event loop use chala deta hai.

Example:

```
console.log("1");
setTimeout(() => console.log("2 timer"), 0);
setImmediate(() => console.log("3 immediate"));
process.nextTick(() => console.log("4 nextTick"));
```

```
console.log("5");
// 1, 5, 4, 2, 3 (approx order)
```

Why is it used?

It is used so that the server can handle thousands of requests without creating a new thread for each one.

Interview Answer:

The event loop is what allows Node.js to handle asynchronous, non-blocking operations on a single thread. It moves completed callbacks from the queues into the call stack whenever the stack is empty.

Important Point:

- process.nextTick and promises run before timers.
- Blocking the loop with heavy synchronous code slows down the whole server.

Q15. What is asynchronous programming?

Easy English:

Asynchronous programming means the program does not wait for a slow task to finish.

- It starts the task, continues with other code, and handles the result later.
- Results are handled by callbacks, Promises or async/await.

Hinglish:

Asynchronous programming ka matlab hai program slow kaam ke liye rukta nahi. Kaam shuru karke aage badh jaata hai aur result aane par callback, Promise ya async/await se handle karta hai.

Example:

```
const fs = require("fs");

fs.readFile("data.txt", "utf8", (err, data) => {
  if (err) return console.log(err);
  console.log("File:", data);
});
console.log("This runs first");
```

Why is it used?

It is used for API calls, database queries and file operations so that the server stays free to serve other users.

Interview Answer:

Asynchronous programming means the code does not wait for a slow task to complete. The task runs in the background and its result is handled later using callbacks, Promises or async/await.

Important Point:

- Node.js is asynchronous and non-blocking by default.
- Async does not mean multi-threaded.

Q16. What is synchronous programming?

Easy English:

Synchronous programming means code runs line by line, and each line waits for the previous one.

- If one task is slow, everything after it is blocked.
- In Node.js, methods ending with Sync are synchronous.

Hinglish:

Synchronous programming me code ek ek line karke chalta hai aur har line pichhli line ka wait karta hai. Agar ek kaam slow hai to baaki sab ruk jaata hai. Node me Sync wale methods aise hi hote hain.

Example:

```
const fs = require("fs");

const data = fs.readFileSync("data.txt", "utf8");  // waits here
```

```
console.log(data);
console.log("This runs after the file is read");
```

Why is it used?

It is used for small startup tasks like reading a config file once, where waiting is acceptable.

Interview Answer:

Synchronous programming executes code line by line, where each line waits for the previous one to finish. In Node.js, synchronous methods block the event loop, so we avoid them in request handlers.

Important Point:

- readFileSync blocks the server for every user, not just one.
- Use synchronous code only at server startup.

Q17. Difference between synchronous and asynchronous.

Easy English:

Synchronous code waits for each task, asynchronous code does not.

- Synchronous is simple but blocking.
- Asynchronous is non-blocking and better for servers.

Hinglish:

Synchronous code har kaam ka wait karta hai, asynchronous nahi karta. Synchronous simple hota hai lekin server ko block kar deta hai, isliye Node.js me async tarika use karte hain.

Example:

```
// Synchronous - blocks
const d = fs.readFileSync("a.txt", "utf8");

// Asynchronous - does not block
fs.readFile("a.txt", "utf8", (e, d) => console.log(d));
```

Why is it used?

We need to know this because using synchronous code inside an API route can slow down the entire application.

Interview Answer:

Synchronous code runs line by line and blocks the thread until each task finishes. Asynchronous code starts a task and continues, handling the result later with callbacks or async/await. Node.js prefers asynchronous for performance.

Important Point:

- Blocking code freezes the event loop for all users.
- Async improves throughput, not raw speed of a single task.

Point	Synchronous	Asynchronous
Execution	Line by line	Does not wait
Blocking	Yes	No
Example	readFileSync	readFile, fetch
Handling result	Direct return	Callback, Promise, await
Good for	Small startup tasks	APIs, DB, file I/O

Q18. What is CommonJS?

Easy English:

CommonJS is the default module system of Node.js.

- We import with require().
- We export with module.exports.
- Loading happens synchronously at runtime.

Hinglish:

CommonJS Node.js ka default module system hai. Isme require() se import karte hain aur module.exports se export. Ye synchronously load hota hai.

Example:

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };

// app.js
const { add } = require("./math");
console.log(add(2, 3));   // 5
```

Why is it used?

It is used in most existing Node.js projects and tutorials because it was the standard before ES Modules arrived.

Interview Answer:

CommonJS is the default module system in Node.js. It uses require to import and module.exports to export, and modules are loaded synchronously.

Important Point:

- CommonJS is Node-specific, it does not run in the browser.
- You cannot use top-level await in CommonJS.

Q19. What are ES Modules?

Easy English:

ES Modules (ESM) are the official JavaScript module system.

- We use import and export.
- They work in browsers and in Node.js.
- In Node we enable them with "type": "module" in package.json or the .mjs extension.
- They support top-level await and tree shaking.

Hinglish:

ES Modules JavaScript ka official module system hai jisme import aur export use hote hain. Browser aur Node dono me chalta hai. Node me chalane ke liye package.json me "type": "module" likhna padta hai.

Example:

```
// math.js
export function add(a, b) { return a + b; }

// app.js
import { add } from "./math.js";
console.log(add(2, 3));
```

Why is it used?

They are used in modern projects, especially React, because the syntax is standard and bundlers can remove unused code.

Interview Answer:

ES Modules are the standard JavaScript module system using import and export. They work in both the browser and Node.js, and support features like top-level await and tree shaking.

Important Point:

- In ESM, file extensions are required in relative imports.
- React projects use ES Modules by default.

Q20. Difference between require() and import.

Easy English:

require() belongs to CommonJS and import belongs to ES Modules.

- require is synchronous and can be used anywhere in the file, even inside an if block.
- import is static, must be at the top level, and is asynchronous in nature.

Hinglish:

require() CommonJS ka hai aur import ES Modules ka. require kahin bhi likha ja sakta hai aur synchronously chalta hai. import hamesha file ke top par hota hai aur modern standard hai.

Example:

```
// CommonJS
const express = require("express");
module.exports = app;

// ES Modules
import express from "express";
export default app;
```

Why is it used?

Knowing the difference is important because mixing both systems in one project causes errors.

Interview Answer:

require is the CommonJS way of importing and is synchronous, while import is the ES Module standard and is static and hoisted. Node supports both, but we should use one consistently in a project.

Important Point:

- Do not mix require and import in the same file.
- ESM needs "type": "module" in package.json.

Point	require()	import
Module system	CommonJS	ES Modules
Placement	Anywhere in file	Top level only
Loading	Synchronous	Static, async friendly
Export keyword	module.exports	export / export default
Browser support	No	Yes

Q21. What is fs module?

Easy English:

fs stands for File System. It is a built-in Node module to work with files.

- We can read, write, append, delete and rename files.
- Every method has an async and a Sync version.
- fs.promises gives Promise based methods for async/await.

Hinglish:

fs ka matlab File System hai. Ye Node ka built-in module hai jisse file read, write, delete aur rename kar sakte hain. Har method ka async aur Sync dono version hota hai.

Example:

```
const fs = require("fs");

fs.writeFile("note.txt", "Hello Deepak", (err) => {
  if (err) throw err;
  console.log("File written");
});
```

Why is it used?

It is used for tasks like saving logs, reading config files, handling uploaded files and generating reports.

Interview Answer:

The fs module is Node's built-in File System module used to read, write, update and delete files. It provides both asynchronous and synchronous methods, and we prefer the asynchronous ones on a server.

Important Point:

- Prefer fs.promises with async/await in modern code.
- Avoid Sync methods inside API routes.

Q22. What is HTTP module?

Easy English:

The http module is a built-in Node module used to create a web server.

- It can create a server and handle requests and responses.
- Express is built on top of this module.

Hinglish:

http module Node ka built-in module hai jisse server banate hain. Ye request lekar response bhejta hai. Express bhi andar se isi module ka use karta hai.

Example:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ message: "Hello" }));
});
server.listen(5000);
```

Why is it used?

It is used to handle raw HTTP communication. In real projects we usually use Express because it is simpler.

Interview Answer:

The http module is Node's built-in module for creating servers and handling HTTP requests and responses. Express is a framework built on top of it that makes the work much easier.

Important Point:

- Without Express, routing must be written manually with if conditions.
- The http module supports both server and client requests.

Q23. What is path module?

Easy English:

The path module is a built-in Node module that works with file and folder paths.

- path.join() joins path parts correctly.
- path.resolve() gives an absolute path.
- It handles differences between Windows and Linux path separators.

Hinglish:

path module file aur folder ke paths ke saath kaam karta hai. path.join se paths sahi tarike se judte hain aur Windows/Linux dono par sahi chalte hain, isliye manual slash lagane se better hai.

Example:

```
const path = require("path");

console.log(path.join(__dirname, "uploads", "a.png"));
console.log(path.extname("photo.png"));   // .png
```

Why is it used?

It is used to build safe file paths for uploads, static files and templates, so the code works on any operating system.

Interview Answer:

The path module is used to handle file and directory paths in Node.js. Methods like path.join and path.resolve build correct paths that work on both Windows and Linux.

Important Point:

- __dirname gives the current folder path in CommonJS.
- Using path.join prevents path related bugs across operating systems.

Q24. What is middleware in Node/Express?

Easy English:

Middleware is a function that runs between the request and the final response.

- It has access to req, res and next.
- It can modify the request, stop it, or pass control using next().
- Examples: express.json(), authentication check, logging, error handling.

Hinglish:

Middleware wo function hai jo request aane ke baad aur response jaane se pehle chalta hai. Iske paas req, res aur next hote hain. Ye request ko check ya modify kar sakta hai aur next() se aage bhej deta hai.

Example:

```
const logger = (req, res, next) => {
  console.log(req.method, req.url);
  next();          // pass to the next middleware
};
app.use(logger);
```

Why is it used?

It is used for common tasks like parsing JSON, checking a JWT token, logging requests and handling errors in one place.

Interview Answer:

Middleware is a function that runs between the request and the response in Express. It receives req, res and next, and it is used for tasks like body parsing, authentication, logging and error handling.

Important Point:

- If you forget next(), the request will hang.
- Middleware order matters, it runs top to bottom.

Q25. What is environment variable?

Easy English:

An environment variable is a value stored outside the code, in the environment where the app runs.

- It keeps secrets and settings separate from the source code.
- In Node we read it with process.env.

Hinglish:

Environment variable wo value hai jo code ke bahar rakhi jaati hai, jaise database ka URL ya secret key. Node me hum ise process.env se padhte hain.

Example:

```
# .env file
PORT=5000
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/mydb
JWT_SECRET=mysecretkey
```

Why is it used?

It is used so that the same code can run in development, testing and production with different settings, without changing any line of code.

Interview Answer:

An environment variable is a configuration value stored outside the code, such as a port, database URL or secret key. In Node.js we access them through process.env.

Important Point:

- They keep secrets out of the source code.
- On hosting platforms we set them in the dashboard, not in a file.

Q26. Why do we use .env?

Easy English:

The .env file stores environment variables in one place during development.

- The dotenv package loads it into process.env.
- It keeps passwords, API keys and database URLs out of the code.
- Different environments can use different .env files.

Hinglish:

.env file me hum saari secret values ek jagah rakhte hain. dotenv package inhe process.env me load kar deta hai. Isse password aur API key code me likhne ki zarurat nahi padti.

Example:

```
// install: npm i dotenv
require("dotenv").config();

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log("Running on " + PORT));
```

Why is it used?

It is used to keep the project secure and configurable, and it makes deployment easy because only the values change, not the code.

Interview Answer:

We use a .env file to store secrets and configuration such as the database URL, port and JWT secret. The dotenv package loads them into process.env, so secrets stay out of the code.

Important Point:

- Always call dotenv.config() at the very top of the entry file.
- Add .env to .gitignore.

Q27. Why should .env not be uploaded to GitHub?

Easy English:

Because it contains secret information.

- It has database passwords, JWT secrets and API keys.
- If the repo is public, anyone can steal the data or misuse paid services.
- Even in a private repo, secrets stay in Git history forever.
- We add it to .gitignore and share a sample file called .env.example instead.

Hinglish:

.env me database password, JWT secret aur API keys hoti hain. Agar ye GitHub par chali gayi to koi bhi database access kar sakta hai ya paid service ka misuse kar sakta hai. Isliye .env ko .gitignore me daalte hain aur team ko .env.example dete hain.

Example:

```
# .gitignore
.env
```

```
node_modules

# .env.example (safe to commit)
PORT=
MONGO_URI=
JWT_SECRET=
```

Why is it used?

It is done for security. Leaked credentials are one of the most common reasons for hacked databases.

Interview Answer:

We never push .env to GitHub because it contains secrets like database passwords and JWT keys. Anyone who sees them can access our database, so we add .env to .gitignore and commit only a .env.example file.

Important Point:

- If a secret is leaked, rotate (change) it immediately.
- Hosting platforms store secrets in their own environment settings.

Q28. What is server.listen()?

Easy English:

listen() starts the server and makes it wait for incoming requests on a given port.

- It takes a port number and an optional callback.
- Without it, the server code exists but never accepts requests.

Hinglish:

listen() server ko start karta hai aur ek port par requests ka intezaar karvata hai. Isme port number dete hain aur ek callback jo server start hone par chalta hai. Iske bina server chalu hi nahi hota.

Example:

```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Why is it used?

It is used to make the application reachable so that the browser or Postman can send requests to it.

Interview Answer:

server.listen() starts the server and binds it to a port so it can accept incoming requests. We usually take the port from process.env.PORT with a default fallback.

Important Point:

- EADDRINUSE error means the port is already in use.
- Hosting platforms provide their own port through process.env.PORT.

Q29. What is a port?

Easy English:

A port is a number that identifies a specific application on a computer.

- The IP address finds the machine, the port finds the app on that machine.
- Common ports: 3000 and 5173 for React dev servers, 5000 for Node APIs, 27017 for MongoDB.

Hinglish:

Port ek number hai jo batata hai ki ek computer par kaunsa application chal raha hai. IP se machine milti hai aur port se us machine ka sahi app. Jaise React 3000/5173 par, Node API 5000 par, MongoDB 27017 par.

Example:

```
app.listen(5000);
// open in browser:  http://localhost:5000
```

Why is it used?

Ports are used so that many applications can run on the same computer at the same time without clashing.

Interview Answer:

A port is a number that identifies a particular application running on a machine. The IP address locates the computer and the port locates the specific service, like 5000 for our Node API.

Important Point:

- Two apps cannot use the same port together.
- Frontend and backend on different ports need CORS.

Q30. What is npm script?

Easy English:

npm scripts are shortcut commands written in the scripts section of package.json.

- We run them with npm run <name>.
- start and test can be run without the word run.
- They are used for starting the server, building the app and running tests.

Hinglish:

npm script package.json ke scripts section me likhe short commands hote hain. Inhe npm run naam se chalate hain. Isse lamba command yaad rakhne ki zarurat nahi padti.

Example:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js",
  "build": "vite build"
}

// run:  npm run dev
```

Why is it used?

They are used to keep all project commands in one place so that any developer can run the project the same way.

Interview Answer:

npm scripts are custom commands defined in package.json and run with npm run. They are used for tasks like

PART 3

EXPRESS.JS

PART 3 - EXPRESS.JS

Express.js (Q1 - Q30)

Q1. What is Express.js?

Easy English:

Express.js is a lightweight web framework built on top of Node.js.

- It makes it easy to create servers, routes and REST APIs.
- It provides routing, middleware support and helper methods.
- It is the "E" in the MERN stack.

Hinglish:

Express.js Node.js ke upar bana ek halka framework hai. Isse server, routes aur REST API banana bahut aasan ho jaata hai. MERN ka "E" isi ko kehte hain.

Example:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => res.send("Hello"));
app.listen(5000);
```

Why is it used?

It is used because writing a server with the raw http module needs a lot of manual code for routing and parsing.

Interview Answer:

Express.js is a minimal web framework for Node.js. It simplifies routing, middleware and request handling, and it is the standard way to build REST APIs in the MERN stack.

Important Point:

- Express is unopinionated, we decide the folder structure.
- It is built on top of Node's http module.

Q2. Why do we use Express?

Easy English:

We use Express because:

- Routing becomes simple with app.get, app.post and so on.
- Middleware support makes auth, logging and validation easy.
- Built-in helpers like res.json() and express.json() save time.
- It has a huge ecosystem of packages like cors, morgan and multer.

Hinglish:

Express isliye use karte hain kyunki routing bahut simple ho jaati hai, middleware se authentication aur validation aasan hoti hai, aur res.json jaise helpers time bachate hain. Bina Express ke ye sab manually likhna padta.

Example:

```
app.use(express.json());
app.post("/api/users", (req, res) => {
  const { name } = req.body;
  res.status(201).json({ message: "Created", name });
});
```

Why is it used?

It is used to build REST APIs quickly with clean and organised code.

Interview Answer:

We use Express because it gives simple routing, middleware support and useful helpers, so we can build REST APIs quickly instead of handling raw HTTP requests manually.

Important Point:

- Express is the most used Node.js framework.
- It works well with MongoDB and Mongoose in the MERN stack.

Q3. What is an Express server?

Easy English:

An Express server is a Node application created with express() that listens on a port and answers HTTP requests.

- We create an app, add middleware, define routes and call listen().

Hinglish:

Express server ek Node application hai jo express() se banta hai, ek port par sunta hai aur requests ka jawab deta hai. Pehle app banate hain, middleware lagate hain, routes likhte hain, phir listen() karte hain.

Example:

```
const express = require("express");
const app = express();

app.use(express.json());
app.get("/api/health", (req, res) => res.json({ ok: true }));

app.listen(5000, () => console.log("Server started"));
```

Why is it used?

It is used as the backend of a MERN application, serving data to the React frontend.

Interview Answer:

An Express server is created using the express() function. We add middleware, define routes, and start it with app.listen so it can handle HTTP requests on a port.

Important Point:

- express.json() must come before routes that read req.body.
- The typical entry file is server.js or index.js.

Q4. What is a route?

Easy English:

A route defines what should happen when a specific URL and HTTP method are requested.

- It has a method (GET, POST, PUT, PATCH, DELETE), a path and a handler function.
- Routes can take dynamic parameters like /users/:id.

Hinglish:

Route batata hai ki kaunse URL aur kaunse method par kya kaam hoga. Isme method, path aur handler function hota hai. Path me :id jaisa dynamic part bhi ho sakta hai.

Example:

```
app.get("/api/users", (req, res) => res.json(users));

app.get("/api/users/:id", (req, res) => {
  res.json({ id: req.params.id });
});
```

Why is it used?

Routes are used to organise the API so each URL does one clear job.

Interview Answer:

A route in Express maps an HTTP method and URL path to a handler function. For example app.get("/api/users") returns the user list, and /api/users/:id returns one user.

Important Point:

- Route order matters, specific routes should come before general ones.
- REST APIs use plural nouns like /api/products.

Q5. What is Router?

Easy English:

express.Router() creates a mini application that holds a group of related routes.

- It keeps route files separate from server.js.
- We connect it to the main app using app.use().

Hinglish:

express.Router() ek chhota app banata hai jisme related routes rakhte hain. Isse har feature ki routes alag file me aa jaati hain aur server.js saaf rehta hai. Phir app.use() se ise jodte hain.

Example:

```
// routes/userRoutes.js
const router = require("express").Router();
router.get("/", getUsers);
router.post("/", createUser);
module.exports = router;

// server.js
app.use("/api/users", require("./routes/userRoutes"));
```

Why is it used?

It is used to keep a project modular and easy to maintain when there are many routes.

Interview Answer:

express.Router() is used to create modular route files. We group related routes in a router and mount it on the main app with app.use, which keeps the project organised.

Important Point:

- Routers can have their own middleware.
- This is the base of the MVC folder structure in Express projects.

Q6. What is middleware?

Easy English:

Middleware is a function that runs in the middle of the request-response cycle.

- Signature: (req, res, next).
- It can read or change req and res.
- It calls next() to pass control, or ends the request with a response.
- Types: application level, router level, built-in, third-party and error handling.

Hinglish:

Middleware wo function hai jo request aur response ke beech me chalta hai. Iske paas req, res aur next hote hain. Ye request check ya modify karke next() se aage bhej deta hai, ya khud response bhej deta hai.

Example:

```
const auth = (req, res, next) => {
  if (!req.headers.authorization)
    return res.status(401).json({ message: "No token" });
  next();
};

app.get("/api/profile", auth, (req, res) => res.json(user));
```

Why is it used?

It is used for shared tasks like body parsing, authentication, logging, CORS and error handling.

Interview Answer:

Middleware is a function with access to req, res and next that runs between the request and the response. It is used for parsing, authentication, logging and error handling in Express.

Important Point:

- Middleware runs in the order it is written.
- Forgetting next() makes the request hang forever.

Q7. Why do we use middleware?

Easy English:

We use middleware to avoid repeating the same code in every route.

- Common checks are written once and applied to many routes.
- Examples: parsing JSON, verifying a JWT token, logging requests, handling errors.

Hinglish:

Middleware isliye use karte hain taaki same code har route me na likhna pade. Ek baar likh kar bahut saare routes par laga dete hain, jaise token verify karna ya request log karna.

Example:

```
app.use(express.json());          // parse JSON body
app.use(cors());                    // allow frontend
app.use(morgan("dev"));             // log requests

router.get("/orders", verifyToken, getOrders);   // protect route
```

Why is it used?

It is used to keep code DRY, secure and easy to maintain, with cross-cutting logic in one place.

Interview Answer:

We use middleware to handle common tasks in one place, such as parsing the request body, checking authentication and logging. It keeps route handlers clean and avoids repeated code.

Important Point:

- Global middleware uses app.use, route-level middleware is passed in the route.
- Error middleware must be added at the end.

Q8. What is app.use()?

Easy English:

app.use() is used to register middleware or mount a router.

- If no path is given, it runs for every request.
- If a path is given, it runs only for URLs starting with that path.

Hinglish:

app.use() middleware register karne ya router jodne ke liye hota hai. Agar path nahi diya to har request par chalta hai, aur path dene par sirf usi path ki requests par chalta hai.

Example:

```
app.use(express.json());                // all requests
app.use("/api/users", userRoutes);        // only /api/users/*
app.use(express.static("public"));        // serve static files
```

Why is it used?

It is used to apply middleware globally and to organise routes by feature.

Interview Answer:

app.use() registers middleware or mounts a router in Express. Without a path it runs for all requests, and with a path it runs only for matching routes.

Important Point:

- Position matters, app.use(express.json()) must come before routes.
- app.use runs for all HTTP methods, unlike app.get.

Q9. What is app.get()?

Easy English:

app.get() defines a route that handles HTTP GET requests.

- GET is used to read or fetch data.
- It should not change data on the server.
- Data is usually sent through params or query, not body.

Hinglish:

app.get() GET request handle karta hai. GET ka kaam data laana hota hai, data badalna nahi. Isme data params ya query se aata hai, body se nahi.

Example:

```
app.get("/api/products", async (req, res) => {
  const products = await Product.find();
  res.status(200).json(products);
});
```

Why is it used?

It is used for listing data, fetching a single record, searching and filtering.

Interview Answer:

app.get() handles GET requests, which are used to read data from the server. GET requests are safe and should not modify any data.

Important Point:

- GET requests can be cached by browsers.
- Never send passwords in a GET query string.

Q10. What is app.post()?

Easy English:

app.post() defines a route that handles HTTP POST requests.

- POST is used to create new data.
- Data is sent in the request body.
- We need express.json() to read JSON body data.
- The correct success status is 201 Created.

Hinglish:

app.post() POST request handle karta hai. POST naya data banane ke liye hota hai aur data req.body me aata hai. Body padhne ke liye express.json() lagana zaroori hai.

Example:

```
app.use(express.json());

app.post("/api/products", async (req, res) => {
  const product = await Product.create(req.body);
  res.status(201).json(product);
});
```

Why is it used?

It is used for registration, login, adding a product, placing an order and any create operation.

Interview Answer:

app.post() handles POST requests, which are used to create new resources. The data comes in req.body, and we usually respond with status 201 after creating it.

Important Point:

- Without express.json(), req.body is undefined.
- POST is not idempotent, calling it twice creates two records.

Q11. What is app.put()?

Easy English:

app.put() handles PUT requests, which update an existing resource.

- PUT replaces the whole document with the new data.
- Fields not sent may be removed or reset.
- It is idempotent: sending the same request twice gives the same result.

Hinglish:

app.put() PUT request handle karta hai jo poora record update karta hai. Isme jo fields nahi bheji gayi wo hat sakti hain, kyunki PUT poora replace karta hai.

Example:

```
app.put("/api/products/:id", async (req, res) => {
  const updated = await Product.findByIdAndUpdate(
    req.params.id, req.body, { new: true }
  );
  res.json(updated);
});
```

Why is it used?

It is used when the client sends the complete updated object, for example a full profile edit form.

Interview Answer:

app.put() handles PUT requests and is used for a full update of a resource. It replaces the existing data with the data sent in the request body.

Important Point:

- PUT is idempotent.
- Use PUT when the whole object is sent, PATCH when only some fields are sent.

Q12. What is app.patch()?

Easy English:

app.patch() handles PATCH requests, which update only some fields.

- Only the fields sent in the body are changed.
- Other fields stay as they are.

Hinglish:

app.patch() PATCH request handle karta hai jo sirf kuch fields update karta hai. Jo fields bheji hain wahi badalti hain, baaki data waisa hi rehta hai.

Example:

```
app.patch("/api/products/:id", async (req, res) => {
  const updated = await Product.findByIdAndUpdate(
    req.params.id, { $set: req.body }, { new: true }
  );
  res.json(updated);
});
```

Why is it used?

It is used for small updates like changing only the status, price or password.

Interview Answer:

app.patch() handles PATCH requests and is used for a partial update. Only the fields sent in the body are modified, which is lighter than PUT.

Important Point:

- PATCH sends less data over the network.
- Mongoose findByIdAndUpdate with $set fits PATCH well.

Q13. What is app.delete()?

Easy English:

app.delete() handles DELETE requests, which remove a resource.

- The id usually comes from req.params.
- Common responses are 200 with a message or 204 with no content.
- If the record is not found we return 404.

Hinglish:

app.delete() DELETE request handle karta hai jo record hata deta hai. Id aam taur par req.params se aati hai. Record na mile to 404 bhejte hain.

Example:

```
app.delete("/api/products/:id", async (req, res) => {
  const p = await Product.findByIdAndDelete(req.params.id);
  if (!p) return res.status(404).json({ message: "Not found" });
  res.json({ message: "Deleted" });
});
```

Why is it used?

It is used to remove records such as deleting a post, a cart item or a user account.

Interview Answer:

app.delete() handles DELETE requests used to remove a resource, usually identified by an id in req.params. We return 404 if the record does not exist.

Important Point:

- Protect delete routes with authentication and authorization.
- Soft delete (an isDeleted flag) is often safer than permanent delete.

Q14. Difference between PUT and PATCH.

Easy English:

Both update data, but the amount is different.

- PUT does a full update and replaces the whole resource.
- PATCH does a partial update and changes only the given fields.

Hinglish:

Dono update karte hain, fark itna hai ki PUT poora record replace karta hai aur PATCH sirf bheje gaye fields badalta hai. Chhote update ke liye PATCH better hai.

Example:

```
// PUT - send the whole object
{ "name": "Laptop", "price": 50000, "stock": 5 }

// PATCH - send only what changes
{ "price": 45000 }
```

Why is it used?

Choosing the right one avoids accidentally erasing fields that were not sent in a PUT request.

Interview Answer:

PUT is used for a full update and replaces the entire resource, while PATCH is used for a partial update and modifies only the fields sent. PUT is idempotent, PATCH usually is too but not always.

Important Point:

- A PUT without all fields can wipe existing data.
- PATCH is preferred for small edits like status changes.

Point	PUT	PATCH
Update type	Full replace	Partial update
Data sent	Whole object	Only changed fields
Missing fields	May be removed	Stay unchanged
Idempotent	Yes	Usually yes
Best for	Full edit form	Status or single field change

Q15. What are req and res?

Easy English:

req (request) is the object that holds all information coming from the client.

- It contains params, query, body, headers and cookies.
- res (response) is the object used to send data back.
- res has methods like res.json(), res.send() and res.status().

Hinglish:

req me client se aayi saari information hoti hai jaise params, query, body aur headers. res se hum client ko jawab bhejte hain, jaise res.json ya res.send.

Example:

```
app.post("/api/login", (req, res) => {
  console.log(req.body);        // data from client
  console.log(req.headers);     // headers
  res.status(200).json({ token: "abc" });
});
```

Why is it used?

They are used for two-way communication between the frontend and the backend in every route.

Interview Answer:

req is the request object containing data sent by the client such as params, query, body and headers. res is the response object used to send data back using methods like res.json and res.status.

Important Point:

- Only one response should be sent per request.
- Sending two responses causes "Cannot set headers after they are sent".

Q16. What is req.params?

Easy English:

req.params contains route parameters from the URL path.

- Defined with a colon, for example /users/:id.
- The values are always strings.

Hinglish:

req.params me URL ke path wale dynamic values aate hain, jaise /users/:id me id. Ye hamesha string hoti hai, isliye number chahiye to convert karna padta hai.

Example:

```
// GET /api/users/65f1a2b3
app.get("/api/users/:id", (req, res) => {
  console.log(req.params.id);   // "65f1a2b3"
  res.json({ id: req.params.id });
});
```

Why is it used?

It is used to identify one specific resource, such as one user, one product or one order.

Interview Answer:

req.params holds the dynamic values from the URL path, defined with a colon like /users/:id. It is mainly used to fetch, update or delete a single record.

Important Point:

- Values are strings, convert with Number() when needed.
- Params are required, query parameters are optional.

Q17. What is req.query?

Easy English:

req.query contains the values written after the question mark in the URL.

- Format: /products?category=mobile&page=2.
- All values come as strings.
- Query parameters are optional.

Hinglish:

req.query me URL ke question mark ke baad wali values aati hain, jaise ?category=mobile&page=2. Ye optional hoti hain aur string me milti hain.

Example:

```
// GET /api/products?category=mobile&page=2
app.get("/api/products", (req, res) => {
  const { category, page } = req.query;
  res.json({ category, page });
});
```

Why is it used?

It is used for searching, filtering, sorting and pagination.

Interview Answer:

req.query contains the query string values from the URL after the question mark. It is used for optional data like filters, search text, sorting and pagination.

Important Point:

- Query values are strings, so parse numbers before using them.
- Always validate query values to avoid injection issues.

Q18. What is req.body?

Easy English:

req.body contains data sent by the client in the request body.

- Used in POST, PUT and PATCH requests.
- It needs express.json() middleware to read JSON data.
- It is safer for sensitive data because it is not visible in the URL.

Hinglish:

req.body me wo data hota hai jo client body me bhejta hai, jaise form ka data. POST, PUT aur PATCH me use hota hai. Ise padhne ke liye express.json() lagana zaroori hai.

Example:

```
app.use(express.json());

app.post("/api/register", (req, res) => {
  const { name, email, password } = req.body;
  res.status(201).json({ name, email });
});
```

Why is it used?

It is used to send form data, login details and JSON objects from the frontend to the backend.

Interview Answer:

req.body holds the data sent in the body of a POST, PUT or PATCH request. We must use express.json() middleware to parse JSON, otherwise req.body will be undefined.

Important Point:

- GET requests normally do not have a body.
- Use express.urlencoded() for HTML form data.

Q19. Difference between params, query and body.

Easy English:

All three carry data from the client, but in different places.

- params: part of the URL path, used to identify a resource.
- query: after the question mark, used for optional filters.
- body: inside the request body, used to send data for create or update.

Hinglish:

Teeno client ka data laate hain lekin jagah alag hai. params URL ke path me hota hai aur ek record identify karta hai. query filter ke liye hoti hai. body me create ya update ka data aata hai.

Example:

```
// params: /api/users/101
// query:  /api/users?role=admin&page=2
// body:   { "name": "Deepak", "email": "d@mail.com" }

app.get("/api/users/:id", (req, res) => {
  console.log(req.params.id, req.query.role, req.body);
});
```

Why is it used?

Knowing the difference helps design clean REST APIs and read data correctly on the server.

Interview Answer:

req.params comes from the URL path and identifies a resource, req.query comes after the question mark and is used for filters and pagination, and req.body carries the data sent in POST, PUT or PATCH requests.

Important Point:

- params and query are visible in the URL, body is not.
- Passwords must always go in the body, never in the query.

Point	req.params	req.query	req.body
Where	URL path	After ? in URL	Request body
Example	/users/:id	?page=2	JSON object
Used with	GET, PUT, DELETE	Mostly GET	POST, PUT, PATCH
Purpose	Identify one record	Filter, sort, paginate	Send data
Visible in URL	Yes	Yes	No

Q20. What is res.json()?

Easy English:

res.json() sends a JSON response to the client.

- It automatically sets Content-Type to application/json.
- It converts objects and arrays to a JSON string.
- It is the standard way to reply in a REST API.

Hinglish:

res.json() JSON response bhejta hai. Ye khud Content-Type application/json set kar deta hai aur object ko JSON string me convert kar deta hai. REST API me yahi standard tarika hai.

Example:

```
app.get("/api/user", (req, res) => {
  res.json({ id: 1, name: "Deepak" });
});
```

Why is it used?

It is used because frontend frameworks like React expect JSON data from the API.

Interview Answer:

res.json() sends a JSON response and automatically sets the correct Content-Type header. It is the standard way to return data from a REST API.

Important Point:

- res.json() handles objects and arrays properly.
- Call it only once per request.

Q21. What is res.send()?

Easy English:

res.send() sends a response of any type.

- It can send a string, HTML, buffer, object or array.
- It sets the Content-Type automatically based on the data.
- If we send an object, it behaves like res.json().

Hinglish:

res.send() kisi bhi type ka response bhej sakta hai - string, HTML ya object. Ye Content-Type khud decide karta hai. Object bhejne par ye res.json jaisa hi kaam karta hai.

Example:

```
res.send("Hello");                    // text/html
res.send("<h1>Hi</h1>");                // HTML
res.send({ ok: true });                 // JSON
```

Why is it used?

It is used for simple text or HTML responses, while res.json is preferred for APIs.

Interview Answer:

res.send() sends a response of any type and sets the Content-Type automatically. For REST APIs we prefer res.json() because it always returns JSON clearly.

Important Point:

- res.json is more explicit for APIs.
- res.end() sends a response without any data.

Q22. What is res.status()?

Easy English:

res.status() sets the HTTP status code of the response.

- It is chained with send or json.
- Common codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error.

Hinglish:

res.status() response ka status code set karta hai, jaise 200 success ke liye aur 404 not found ke liye. Ise json ya send ke saath chain karte hain, taki frontend sahi tarah handle kar sake.

Example:

```
res.status(201).json({ message: "Created" });
res.status(400).json({ message: "Invalid data" });
res.status(404).json({ message: "User not found" });
```

Why is it used?

It is used so the frontend knows whether the request succeeded or failed and can show the right message.

Interview Answer:

res.status() sets the HTTP status code of a response. We chain it with res.json, for example res.status(201).json() after creating a record and 404 when a record is not found.

Important Point:

- 2xx success, 4xx client error, 5xx server error.
- Always return after sending a response to avoid sending twice.

Q23. What is error-handling middleware?

Easy English:

Error-handling middleware is a special middleware with four parameters: (err, req, res, next).

- Express recognises it by the four parameters.
- It must be added after all routes.
- We send an error to it by calling next(err) or by throwing inside async code.

Hinglish:

Error-handling middleware me chaar parameters hote hain - err, req, res, next. Isko sabhi routes ke baad likhte hain. Isse saare errors ek hi jagah handle ho jaate hain.

Example:

```
// must be last
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500).json({
    message: err.message || "Server Error"
  });
});
```

Why is it used?

It is used to handle all errors in one place so that routes stay clean and the client always gets a proper JSON error.

Interview Answer:

Error-handling middleware in Express takes four arguments (err, req, res, next) and is defined after all routes. It catches errors passed with next(err) and sends a consistent error response.

Important Point:

- The four-parameter signature is what makes it an error handler.
- Never send stack traces to users in production.

Q24. What is CORS?

Easy English:

CORS means Cross-Origin Resource Sharing.

- It is a browser security rule.
- By default a browser blocks requests from one origin (domain, port or protocol) to another.
- The server must send headers that allow the other origin.

Hinglish:

CORS ka matlab Cross-Origin Resource Sharing hai. Browser by default ek origin se doosre origin ki request block kar deta hai. Server ko headers bhejkar permission deni padti hai.

Example:

```
// React on 5173, API on 5000 -> different origin
// browser blocks it without CORS headers
```

Why is it used?

It exists for security, so that a random website cannot secretly call another site's API using a logged-in user's session.

Interview Answer:

CORS is a browser security mechanism that blocks requests between different origins unless the server allows them through response headers. In MERN it appears because React and the API run on different ports.

Important Point:

- CORS is enforced by the browser, not by Postman.
- Origin = protocol + domain + port.

Q25. Why do we use CORS?

Easy English:

We use the cors package so our React frontend can call our Express API.

- It adds the required Access-Control-Allow-Origin headers.
- We can allow all origins during development and only our domain in production.

Hinglish:

Hum cors package isliye lagate hain taaki React frontend hamari Express API ko call kar sake. Ye zaroori headers add kar deta hai. Development me sab allow kar sakte hain, production me sirf apna domain.

Example:

```
const cors = require("cors");

app.use(cors());                          // allow all (development)

app.use(cors({                            // safer (production)
  origin: "https://myapp.com",
  credentials: true
}));
```

Why is it used?

It is used to fix the common "blocked by CORS policy" error in MERN projects.

Interview Answer:

We use CORS so that our frontend running on a different port or domain can call our backend API. The cors middleware adds the required headers to allow those requests.

Important Point:

- Allowing all origins in production is a security risk.
- For cookies you need credentials: true on both sides.

Q26. What is authentication?

Easy English:

Authentication answers the question "Who are you?".

- It verifies the identity of a user, usually by email and password.
- After success, the server issues a token or session.
- Passwords must be stored hashed, never as plain text.

Hinglish:

Authentication ka matlab hai "Aap kaun ho?" ye verify karna. Email aur password check karke user ki pehchaan confirm hoti hai, phir token milta hai. Password hamesha hash karke store karte hain.

Example:

```
const bcrypt = require("bcryptjs");

const user = await User.findOne({ email });
const ok = await bcrypt.compare(password, user.password);
if (!ok) return res.status(401).json({ message: "Invalid credentials" });
```

Why is it used?

It is used to make sure only registered and valid users can enter the application.

Interview Answer:

Authentication is the process of verifying who a user is, usually with email and password. On success the server returns a JWT token that the client sends with future requests.

Important Point:

- Always hash passwords with bcrypt before saving.
- Failed authentication returns status 401.

Q27. What is authorization?

Easy English:

Authorization answers the question "What are you allowed to do?".

- It happens after authentication.
- It checks roles and permissions, for example admin or normal user.
- If not allowed, the server returns 403 Forbidden.

Hinglish:

Authorization ka matlab hai "Aap kya kar sakte ho?". Ye authentication ke baad hota hai. Isme role check hota hai jaise admin ya normal user. Permission na ho to 403 milta hai.

Example:

```
const isAdmin = (req, res, next) => {
  if (req.user.role !== "admin")
    return res.status(403).json({ message: "Access denied" });
  next();
};

router.delete("/api/users/:id", verifyToken, isAdmin, deleteUser);
```

Why is it used?

It is used to protect sensitive actions so that only the right users, such as admins, can perform them.

Interview Answer:

Authorization decides what an authenticated user is allowed to do, based on their role or permissions. It comes after authentication and returns 403 when access is denied.

Important Point:

- Authentication comes first, authorization second.

- 401 means not logged in, 403 means logged in but not allowed.

Point	Authentication	Authorization
Question	Who are you?	What can you do?
When	First	After authentication
Checks	Email and password	Role and permissions
Status code	401 Unauthorized	403 Forbidden
Example	Login	Only admin can delete

Q28. What is JWT?

Easy English:

JWT means JSON Web Token. It is a secure token used for stateless authentication.

- It has three parts: header, payload and signature, separated by dots.
- The payload holds data such as the user id and role.
- The signature is created with a secret key, so the token cannot be changed without detection.

Hinglish:

JWT ka matlab JSON Web Token hai. Isme teen hisse hote hain - header, payload aur signature. Payload me user ki id hoti hai aur signature secret key se banta hai, isliye koi token ke saath chhed-chhaad nahi kar sakta.

Example:

```
const jwt = require("jsonwebtoken");

const token = jwt.sign({ id: user._id, role: user.role },
                       process.env.JWT_SECRET, { expiresIn: "7d" });

const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

Why is it used?

It is used to keep a user logged in without storing session data on the server.

Interview Answer:

JWT is a JSON Web Token made of header, payload and signature. The server signs it with a secret key at login, and the client sends it back with each request so the server can verify the user without storing sessions.

Important Point:

- The payload is only encoded, not encrypted, so never put a password in it.
- Always set an expiry time with expiresIn.

Q29. Why do we use JWT?

Easy English:

We use JWT because:

- It is stateless, so the server does not store sessions and can scale easily.
- The token itself carries the user information.
- It works well with mobile apps and multiple servers.
- It is sent in the Authorization header as "Bearer token".

Hinglish:

JWT isliye use karte hain kyunki server ko session yaad nahi rakhna padta, token me hi user ki jaankari hoti hai. Isse scaling aasan hoti hai aur mobile apps ke saath bhi achha chalta hai.

Example:

```
// client sends
// Authorization: Bearer <token>
```

```
const verifyToken = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ message: "No token" });
  req.user = jwt.verify(token, process.env.JWT_SECRET);
  next();
};
```

Why is it used?

It is used for login systems in MERN apps because React can store the token and send it with every API call.

Interview Answer:

We use JWT because it gives stateless authentication. The server does not need to store sessions, the token itself carries user data, and it works across multiple servers and devices.

Important Point:

- Store tokens carefully, httpOnly cookies are safer than localStorage.
- JWTs cannot be cancelled easily before expiry, so keep expiry short.

Q30. What is MVC architecture?

Easy English:

MVC means Model, View, Controller. It is a way to organise code.

- Model: database schema and data logic (Mongoose models).
- View: what the user sees (in MERN this is React).
- Controller: the functions that handle requests and business logic.
- Routes connect URLs to controllers.

Hinglish:

MVC ka matlab Model, View, Controller hai. Model database ka structure hai, View user ko dikhne wala hissa (MERN me React), aur Controller wo function jo request handle karta hai. Isse code organised rehta hai.

Example:

```
// models/User.js      -> schema
// controllers/userController.js -> logic
// routes/userRoutes.js  -> endpoints

// controllers/userController.js
exports.getUsers = async (req, res) => {
  const users = await User.find();
  res.json(users);
};
```

Why is it used?

It is used to separate concerns so the project stays clean, testable and easy for a team to work on.

Interview Answer:

MVC is an architecture that separates the application into Model, View and Controller. In a MERN project, Mongoose

PART 4

REACT.JS

PART 4 - REACT.JS

React.js (Q1 - Q30)

Q1. What is React?

Easy English:

React is an open source JavaScript library for building user interfaces, created by Facebook (Meta).

- It is component based, so the UI is divided into small reusable parts.
- It uses a Virtual DOM for fast updates.
- It is declarative: we describe what the UI should look like and React updates it.

Hinglish:

React ek JavaScript library hai jo user interface banane ke liye Facebook ne banayi. Ye components me kaam karti hai, Virtual DOM use karti hai aur UI ko automatically update karti hai jab data badalta hai.

Example:

```
function App() {
  return <h1>Hello Deepak</h1>;
}
export default App;
```

Why is it used?

It is used to build fast single page applications where the UI updates smoothly when data changes.

Interview Answer:

React is a JavaScript library for building user interfaces. It is component based, uses a Virtual DOM for efficient updates, and follows a declarative approach where the UI updates automatically when state changes.

Important Point:

- React is a library, not a full framework.
- It is the "R" in the MERN stack.

Q2. Why do we use React?

Easy English:

We use React because:

- Components are reusable, which saves time.
- The Virtual DOM makes updates fast.
- One-way data flow makes the app predictable.
- It has a huge ecosystem: React Router, Redux, hooks and many UI libraries.

Hinglish:

React isliye use karte hain kyunki components dobara use ho jaate hain, Virtual DOM ki wajah se update fast hote hain, aur data ek hi direction me chalta hai jisse app predictable rehti hai. Community aur libraries bhi bahut hain.

Example:

```
function Button({ text }) {
  return <button>{text}</button>;
}

// reuse anywhere
<Button text="Save" />
<Button text="Delete" />
```

Why is it used?

It is used for dynamic dashboards, e-commerce sites, social apps and any UI that changes frequently.

Interview Answer:

We use React because it lets us build reusable components, updates the UI efficiently using the Virtual DOM, and has a strong ecosystem for routing and state management.

Important Point:

- React only handles the view layer.
- Routing and state management come from extra libraries.

Q3. What is a component?

Easy English:

A component is a reusable, independent piece of UI.

- It is a JavaScript function that returns JSX.
- Components can receive data through props.
- A page is built by combining many components.

Hinglish:

Component UI ka ek chhota reusable hissa hai. Ye ek JavaScript function hota hai jo JSX return karta hai. Props se data lekar wahi component alag alag jagah use ho sakta hai.

Example:

```
function UserCard({ name, city }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>{city}</p>
    </div>
  );
}
```

Why is it used?

Components are used to break a big UI into small pieces that are easier to build, test and reuse.

Interview Answer:

A component is an independent, reusable piece of UI. In modern React it is a function that returns JSX and can receive data through props.

Important Point:

- Component names must start with a capital letter.
- Function components with hooks are the modern standard.

Q4. What is JSX?

Easy English:

JSX means JavaScript XML. It lets us write HTML-like code inside JavaScript.

- Browsers cannot read JSX directly, so Babel converts it to React.createElement().
- We write JavaScript expressions inside curly braces.
- A component must return a single parent element.

Hinglish:

JSX ka matlab JavaScript XML hai. Isme hum JavaScript ke andar HTML jaisa code likhte hain. Babel isko normal JavaScript me convert karta hai. Curly braces me variable ya expression daal sakte hain.

Example:

```
const name = "Deepak";

const element = (
  <div>
    <h1>Hello {name}</h1>
    <p>{2 + 3}</p>
  </div>
```

```
);
```

Why is it used?

It is used because writing UI with HTML-like syntax is much easier to read than calling createElement again and again.

Interview Answer:

JSX is a syntax extension that lets us write HTML-like code inside JavaScript. Babel compiles it into React.createElement calls, and it makes component code easy to read.

Important Point:

- Use className instead of class and htmlFor instead of for.
- Use a fragment <> </> to return multiple elements without extra divs.

Q5. Why do we use JSX?

Easy English:

We use JSX because:

- It makes the UI code readable and close to HTML.
- Logic and markup stay together in the same component.
- We can insert dynamic values easily using curly braces.
- Errors are caught early during compilation.

Hinglish:

JSX isliye use karte hain kyunki UI likhna HTML jaisa simple lagta hai, logic aur markup ek hi jagah rehte hain, aur dynamic value curly braces se aasani se daal sakte hain.

Example:

```
function Price({ amount }) {
  return <p>Total: {amount > 0 ? amount : "Free"}</p>;
}
```

Why is it used?

It is used to write clean, dynamic UI code instead of long JavaScript function calls.

Interview Answer:

We use JSX because it makes writing UI simple and readable. It combines markup with JavaScript logic and allows dynamic values directly inside the markup.

Important Point:

- JSX is optional in React, but almost everyone uses it.
- Inside JSX we can write expressions, not statements like if.

Q6. What are props?

Easy English:

Props means properties. They are used to pass data from a parent component to a child component.

- Props are read-only, the child cannot change them.
- They make components reusable.
- Data flows only downwards (one-way).

Hinglish:

Props ka matlab properties hai. Inse parent component child ko data bhejta hai. Child props ko badal nahi sakta, sirf padh sakta hai. Isse ek hi component alag data ke saath reuse ho jaata hai.

Example:

```
function Welcome({ name }) {
  return <h2>Welcome {name}</h2>;
}
```

```
// parent
<Welcome name="Deepak" />
<Welcome name="Amit" />
```

Why is it used?

Props are used to make a component dynamic so the same UI can show different data.

Interview Answer:

Props are read-only inputs passed from a parent component to a child component. They make components reusable, and data flows in one direction from parent to child.

Important Point:

- Props are immutable inside the child.
- props.children is used to pass nested JSX content.

Q7. What is state?

Easy English:

State is data that belongs to a component and can change over time.

- When state changes, React re-renders the component.
- We create it with the useState hook in function components.
- It is private to that component.

Hinglish:

State component ka apna data hai jo badal sakta hai. Jab state badalti hai, React component ko dobara render kar deta hai. Function component me useState se state banate hain.

Example:

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Why is it used?

It is used for values that change while the app runs, like form inputs, counters, loading flags and API data.

Interview Answer:

State is data owned by a component that can change over time. When state updates through the setter function, React re-renders the component to show the new value.

Important Point:

- Never change state directly, always use the setter function.
- State updates are asynchronous and batched.

Q8. Difference between props and state.

Easy English:

Both hold data, but ownership is different.

- Props come from the parent and are read-only.
- State is created inside the component and can be changed.

Hinglish:

Dono data rakhte hain, lekin props parent se aate hain aur badle nahi ja sakte, jabki state component ka apna data hai jo badal sakta hai.

Example:

```
function Child({ title }) {            // props
  const [open, setOpen] = useState(false); // state
  return <h3 onClick={() => setOpen(!open)}>{title}</h3>;
}
```

Why is it used?

Knowing the difference helps decide where data should live and how it should flow in the app.

Interview Answer:

Props are read-only data passed from parent to child, while state is data managed inside the component that can change. Both trigger a re-render when their value changes.

Important Point:

- Props flow downward, state stays local.
- Shared data is lifted up to a common parent or put in Context.

Point	Props	State
Source	Parent component	Inside the component
Changeable	No (read-only)	Yes, with setter
Purpose	Configure a component	Track changing data
Owner	Parent	The component itself

Q9. What is useState()?

Easy English:

useState is a React hook that adds state to a function component.

- It returns an array with the current value and a setter function.
- Calling the setter re-renders the component.
- We can pass a function to the setter to use the previous value safely.

Hinglish:

useState ek React hook hai jo function component me state banata hai. Ye do cheezein return karta hai - current value aur usko badalne wala function. Setter chalte hi component dobara render hota hai.

Example:

```
const [name, setName] = useState("");
const [count, setCount] = useState(0);

setCount(prev => prev + 1);   // safe update
```

Why is it used?

It is used for any value that changes in the UI, such as input text, toggles, counters and API data.

Interview Answer:

useState is a hook that lets a function component hold state. It returns the current value and a setter function, and calling the setter re-renders the component with the new value.

Important Point:

- State updates are asynchronous, so use the function form for counters.
- Never mutate state objects directly, create a new object with spread.

Q10. Why do we use useState()?

Easy English:

We use useState because a normal variable does not re-render the UI.

- A normal variable loses its value on each render.
- useState keeps the value between renders and tells React to update the screen.

Hinglish:

useState isliye use karte hain kyunki normal variable badalne se UI update nahi hota aur uski value har render par reset ho jaati hai. useState value ko yaad rakhta hai aur screen ko refresh kar deta hai.

Example:

```
// wrong - UI will not update
let count = 0;
const add = () => { count++; };

// right
const [count, setCount] = useState(0);
const add = () => setCount(count + 1);
```

Why is it used?

It is used to make the UI reactive, so the screen always shows the latest data.

Interview Answer:

We use useState because React needs to know when data changes in order to re-render. A normal variable does not trigger a re-render and its value resets on every render.

Important Point:

- One component can have many useState calls.
- For complex state with many related fields, useReducer is better.

Q11. What is useEffect()?

Easy English:

useEffect is a hook used to perform side effects in a component.

- Side effects are things like API calls, timers, subscriptions and DOM updates.
- It runs after the component renders.
- The second argument is the dependency array which controls when it runs.
- Returning a function from it performs cleanup.

Hinglish:

useEffect side effects ke liye hota hai, jaise API call, timer ya event listener. Ye render hone ke baad chalta hai. Dependency array batata hai ki kab chalna hai, aur return kiya gaya function cleanup karta hai.

Example:

```
useEffect(() => {
  const fetchUsers = async () => {
    const res = await fetch("/api/users");
    setUsers(await res.json());
  };
  fetchUsers();
}, []);          // runs once after first render
```

Why is it used?

It is used to connect a React component with the outside world, mainly to fetch data from an API.

Interview Answer:

useEffect is a hook used to handle side effects like API calls, timers and subscriptions. It runs after render, and the dependency array decides when it runs again.

Important Point:

- An empty array means it runs only once after mount.
- Always clean up timers and listeners in the returned function.

Q12. Why do we use useEffect()?

Easy English:

We use useEffect because side effects should not be written directly in the component body.

- Writing a fetch directly would run on every render and cause an infinite loop.
- useEffect gives control over when the code runs.
- It also handles cleanup when the component unmounts.

Hinglish:

useEffect isliye use karte hain kyunki API call ko seedha component ke andar likhne se har render par wo chalega aur infinite loop ban jaayega. useEffect control deta hai ki kab chale aur cleanup bhi karta hai.

Example:

```
useEffect(() => {
  const id = setInterval(() => console.log("tick"), 1000);
  return () => clearInterval(id);    // cleanup on unmount
}, []);
```

Why is it used?

It is used for data fetching on page load, setting up timers, listening to events and updating the document title.

Interview Answer:

We use useEffect to run side effects at the right time, such as fetching data after the first render. It also lets us clean up timers and listeners when the component unmounts.

Important Point:

- Missing dependencies cause stale data bugs.
- Updating state inside useEffect without a dependency array causes an infinite loop.

Q13. What is dependency array?

Easy English:

The dependency array is the second argument of useEffect.

- It tells React when to run the effect again.
- No array: runs after every render.
- Empty array []: runs only once after the first render.
- [value]: runs whenever that value changes.

Hinglish:

Dependency array useEffect ka doosra argument hai jo batata hai effect kab chalega. Array na ho to har render par, khaali array ho to sirf ek baar, aur value daalne par jab wo value badle tab.

Example:

```
useEffect(() => { ... });          // every render
useEffect(() => { ... }, []);        // once
useEffect(() => { ... }, [userId]);  // when userId changes
```

Why is it used?

It is used to control performance by avoiding unnecessary API calls and re-runs.

Interview Answer:

The dependency array in useEffect controls when the effect runs. An empty array runs it once, listing values runs it when those values change, and omitting it runs it after every render.

Important Point:

- Always include every value used inside the effect.
- Objects and arrays as dependencies change on every render, which can cause extra runs.

Q14. What happens with empty dependency array []?

Easy English:

With an empty array, the effect runs only once, after the first render (mount).

- It does not run again when state or props change.
- The cleanup function runs once when the component unmounts.

Hinglish:

Khaali dependency array dene par effect sirf ek baar chalta hai, pehla render hone ke baad. Baad me state ya props badalne par dobara nahi chalta. Cleanup tab chalta hai jab component hat jaata hai.

Example:

```
useEffect(() => {
  console.log("Runs only once");
  fetchProducts();
  return () => console.log("Cleanup on unmount");
}, []);
```

Why is it used?

It is used for one-time setup work like loading initial data, connecting to a socket or reading from localStorage.

Interview Answer:

An empty dependency array makes the effect run only once after the first render, and its cleanup function runs when the component unmounts. It is mainly used for initial data fetching.

Important Point:

- In React StrictMode during development, effects run twice on purpose.
- Empty array with outdated values inside can cause stale closure bugs.

Q15. What is event handling in React?

Easy English:

Event handling means responding to user actions like click, change and submit.

- React uses camelCase names such as onClick and onChange.
- We pass a function reference, not a string.
- React uses a synthetic event system that works the same in all browsers.

Hinglish:

Event handling matlab user ke click, change ya submit par kuch karna. React me camelCase naam hote hain jaise onClick. Function ka reference dete hain, string nahi. React apna synthetic event system use karta hai.

Example:

```
function Form() {
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("submitted");
  };
  return <button onClick={handleSubmit}>Save</button>;
}
```

Why is it used?

It is used to make the UI interactive: buttons, forms, dropdowns and search boxes.

Interview Answer:

Event handling in React uses camelCase props like onClick and onChange, and we pass a function reference. React wraps browser events in synthetic events for consistent behaviour.

Important Point:

- Use e.preventDefault() to stop a form from reloading the page.
- onClick={fn} passes a reference, onClick={fn()} calls it immediately, which is a common mistake.

Q16. What is controlled component?

Easy English:

A controlled component is a form input whose value is controlled by React state.

- The value comes from state.
- The onChange handler updates the state.
- React is the single source of truth for the input.

Hinglish:

Controlled component wo input hai jiski value React state se aati hai. Value state se milti hai aur onChange se state update hoti hai. Isse React ke paas hamesha latest value rehti hai.

Example:

```
const [email, setEmail] = useState("");

<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

Why is it used?

It is used for form validation, disabling the submit button and showing live feedback while typing.

Interview Answer:

A controlled component is a form element whose value is stored in React state and updated through onChange. React becomes the single source of truth, which makes validation easy.

Important Point:

- An uncontrolled component uses a ref and the DOM keeps the value.
- Forgetting onChange makes the input read-only.

Q17. What is React Hook Form?

Easy English:

React Hook Form is a popular library for handling forms in React.

- It reduces re-renders because it uses uncontrolled inputs internally with refs.
- It gives register, handleSubmit and formState for errors.
- It supports validation rules and schema validation with libraries like Yup or Zod.

Hinglish:

React Hook Form forms handle karne ki ek popular library hai. Ye kam re-render karti hai isliye fast hai. Isme register, handleSubmit aur errors milte hain, aur validation likhna bahut aasan ho jaata hai.

Example:

```
const { register, handleSubmit, formState: { errors } } = useForm();

const onSubmit = (data) => console.log(data);

<form onSubmit={handleSubmit(onSubmit)}>
  <input {...register("email", { required: true })} />
  {errors.email && <span>Email is required</span>}
</form>
```

Why is it used?

It is used in real projects to write less code, get better performance and handle validation cleanly in big forms.

Interview Answer:

React Hook Form is a form library that handles form state and validation with minimal re-renders. It provides register, handleSubmit and formState, which makes large forms simple and fast.

Important Point:

- It performs better than many useState fields in big forms.
- It integrates with Yup or Zod for schema validation.

Q18. What is conditional rendering?

Easy English:

Conditional rendering means showing different UI based on a condition.

- We can use a ternary operator, the && operator or an if statement before the return.

Hinglish:

Conditional rendering ka matlab hai condition ke hisaab se alag alag UI dikhana. Iske liye ternary operator, && ya return se pehle if use karte hain.

Example:

```
{loading ? <Spinner /> : <UserList users={users} />}

{error && <p className="error">{error}</p>}

{isLoggedIn ? <Dashboard /> : <Login />}
```

Why is it used?

It is used to show loaders, error messages, empty states and different views for logged-in and logged-out users.

Interview Answer:

Conditional rendering means displaying different UI based on a condition, using a ternary or the logical AND operator. It is commonly used for loading states, errors and authentication views.

Important Point:

- With && be careful: 0 && <Item /> renders 0 on the screen.
- Return null when nothing should be displayed.

Q19. What is list rendering?

Easy English:

List rendering means displaying an array of data as UI elements.

- We use the map() method to convert data into JSX.
- Each item needs a unique key prop.

Hinglish:

List rendering ka matlab hai array ke data ko UI me dikhana. Iske liye map() use karte hain jo har item ko JSX me badal deta hai. Har item ko ek unique key deni zaroori hai.

Example:

```
{users.map((user) => (
  <li key={user._id}>{user.name}</li>
))}
```

Why is it used?

It is used to show data coming from an API, such as products, posts, comments or search results.

Interview Answer:

List rendering is displaying an array as UI using the map method. Each rendered element must have a unique key so React can track items correctly.

Important Point:

- Use a database id as the key, not the array index.

- forEach does not work here because it does not return anything.

Q20. Why do we use key in React?

Easy English:

The key prop helps React identify which list items changed, were added or removed.

- React compares old and new lists using keys during reconciliation.
- Without keys, React re-renders more than needed and may mix up item state.
- Keys must be unique and stable.

Hinglish:

Key se React samajh paata hai ki list me kaunsa item naya hai, kaunsa hata aur kaunsa badla. Isse update fast hota hai. Key unique aur stable honi chahiye, isliye index ki jagah id use karte hain.

Example:

```
// good
{items.map(item => <Row key={item._id} data={item} />)}

// avoid when the list can change
{items.map((item, i) => <Row key={i} data={item} />)}
```

Why is it used?

It is used to make list updates efficient and to keep the correct state attached to the correct item.

Interview Answer:

React uses the key prop to identify list items during reconciliation. Unique and stable keys make updates efficient and prevent bugs where item state gets attached to the wrong element.

Important Point:

- Index keys cause bugs when items are reordered or deleted.
- Keys must be unique among siblings, not globally.

Q21. What is lifting state up?

Easy English:

Lifting state up means moving shared state to the closest common parent component.

- The parent holds the state and passes it down through props.
- The child updates it by calling a function passed from the parent.

Hinglish:

Lifting state up ka matlab hai jo state do components ko chahiye, use unke common parent me le jaana. Parent state rakhta hai aur props se child ko bhejta hai. Child parent ka function call karke update karta hai.

Example:

```
function Parent() {
  const [text, setText] = useState("");
  return (
    <>
      <SearchBox value={text} onChange={setText} />
      <Results query={text} />
    </>
  );
}
```

Why is it used?

It is used when two sibling components need to share the same data, for example a search box and a result list.

Interview Answer:

Lifting state up means moving state to the nearest common parent so that multiple child components can share it. The parent passes the value and updater down through props.

Important Point:

- Too much lifting causes prop drilling.
- Context API or Redux solves deep prop drilling.

Q22. What is Context API?

Easy English:

Context API is React's built-in way to share data across many components without passing props at every level.

- We create a context, wrap components in a Provider, and read it with useContext.
- It solves prop drilling.

Hinglish:

Context API React ka built-in tarika hai jisse data ko har level par props bheje bina share kiya ja sakta hai. Context banate hain, Provider se wrap karte hain aur useContext se padhte hain. Isse prop drilling khatam ho jaati hai.

Example:

```
const AuthContext = createContext();

function App() {
  const [user, setUser] = useState(null);
  return (
    <AuthContext.Provider value={{ user, setUser }}>
      <Dashboard />
    </AuthContext.Provider>
  );
}
```

Why is it used?

It is used for global data like the logged-in user, theme, language and cart items.

Interview Answer:

Context API lets us share data globally without prop drilling. We create a context, provide a value at the top, and any nested component can read it using useContext.

Important Point:

- Every consumer re-renders when the context value changes.
- For large complex state, Redux Toolkit is often preferred.

Q23. What is useContext()?

Easy English:

useContext is a hook that reads the value from a React context.

- It takes the context object and returns its current value.
- The component must be inside the matching Provider.

Hinglish:

useContext ek hook hai jo context ki value padhta hai. Isme context object dete hain aur current value mil jaati hai. Component Provider ke andar hona chahiye, warna value undefined aayegi.

Example:

```
const { user, setUser } = useContext(AuthContext);

return <p>Welcome {user ? user.name : "Guest"}</p>;
```

Why is it used?

It is used to access global values like the current user or theme directly inside any component.

Interview Answer:

useContext is a hook that reads the current value of a context. It removes the need to pass props through many levels and is used with the Context Provider.

Important Point:

- If the component is outside the Provider, the value will be undefined.
- Splitting contexts reduces unnecessary re-renders.

Q24. What is useRef()?

Easy English:

useRef returns a mutable object with a .current property.

- Changing .current does not cause a re-render.
- It is used to access DOM elements directly.
- It is also used to store values between renders, such as a timer id or a previous value.

Hinglish:

useRef ek object deta hai jisme .current hoti hai. Ise badalne par component re-render nahi hota. Ye DOM element ko pakadne aur renders ke beech value yaad rakhne ke liye use hota hai.

Example:

```
const inputRef = useRef(null);

const focusInput = () => inputRef.current.focus();

<input ref={inputRef} />
<button onClick={focusInput}>Focus</button>
```

Why is it used?

It is used to focus an input, scroll to an element, play a video or keep an interval id without re-rendering.

Interview Answer:

useRef returns a mutable object whose .current value persists across renders without causing a re-render. It is used to access DOM elements and to store values like timer ids.

Important Point:

- Updating a ref never re-renders the component.
- Do not use refs for data that should appear in the UI, use state.

Q25. What is useMemo()?

Easy English:

useMemo caches (memoizes) the result of an expensive calculation.

- It recalculates only when its dependencies change.
- It returns a value.

Hinglish:

useMemo bhaari calculation ka result yaad rakh leta hai aur tabhi dobara calculate karta hai jab dependency badle. Ye value return karta hai, function nahi.

Example:

```
const total = useMemo(() => {
  return products.reduce((sum, p) => sum + p.price, 0);
}, [products]);
```

Why is it used?

It is used to avoid repeating heavy calculations like filtering, sorting or totalling large lists on every render.

Interview Answer:

useMemo memoizes the result of an expensive calculation and recomputes it only when its dependencies change. It helps performance when working with large lists or heavy computations.

Important Point:

- Do not use it everywhere, it adds memory cost.

- useMemo returns a value, useCallback returns a function.

Q26. What is useCallback()?

Easy English:

useCallback caches a function definition between renders.

- Normally a new function is created on every render.
- useCallback returns the same function unless a dependency changes.
- It is mainly used with React.memo child components.

Hinglish:

useCallback function ko yaad rakh leta hai taaki har render par naya function na bane. Jab tak dependency na badle, wahi function milta hai. Ye React.memo wale child components ke saath faydemand hai.

Example:

```
const handleSearch = useCallback((text) => {
  setQuery(text);
}, []);

<SearchBox onSearch={handleSearch} />   // child does not re-render
```

Why is it used?

It is used to prevent unnecessary child re-renders when a function is passed as a prop.

Interview Answer:

useCallback memoizes a function so that the same function reference is kept between renders. It is used with React.memo to avoid unnecessary re-renders of child components.

Important Point:

- useCallback(fn, deps) is the same as useMemo(() => fn, deps).
- Without React.memo on the child, useCallback usually gives no benefit.

Point	useMemo	useCallback
Returns	A value (result)	A function
Purpose	Skip heavy calculation	Keep same function reference
Common use	Filtering, sorting, totals	Props passed to React.memo child
Recomputes	When dependencies change	When dependencies change

Q27. What is React Router?

Easy English:

React Router is a library that adds page navigation to a React single page application.

- It shows different components for different URLs.
- The page does not reload, only the component changes.
- Main parts: BrowserRouter, Routes, Route, Link and useNavigate.

Hinglish:

React Router se React app me alag alag pages banaye jaate hain. URL ke hisaab se alag component dikhta hai aur page reload nahi hota. Isme BrowserRouter, Routes, Route, Link aur useNavigate main cheezein hain.

Example:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/products/:id" element={<Product />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
```

```
</BrowserRouter>
```

Why is it used?

It is used to build multi-page experiences in a single page app with clean URLs and protected routes.

Interview Answer:

React Router is a routing library for React that renders different components based on the URL without reloading the page. It provides BrowserRouter, Routes, Route, Link and hooks like useNavigate.

Important Point:

- Use Link instead of an anchor tag to avoid a full page reload.
- useParams reads URL parameters, useNavigate navigates programmatically.

Q28. What is SPA?

Easy English:

SPA means Single Page Application.

- The browser loads one HTML file, and JavaScript updates the content.
- Navigation does not reload the whole page, which feels fast.
- Data is fetched from APIs in the background.
- Drawback: the first load can be slower and SEO needs extra work.

Hinglish:

SPA ka matlab Single Page Application hai. Ek hi HTML file load hoti hai aur JavaScript content badalta rehta hai. Page reload nahi hota isliye app fast lagti hai, lekin pehla load thoda bhaari ho sakta hai aur SEO ke liye extra kaam karna padta hai.

Example:

```
// index.html has only one root div
<div id="root"></div>
// React renders everything inside it
```

Why is it used?

It is used to give a smooth app-like experience for dashboards, admin panels and social applications.

Interview Answer:

SPA means Single Page Application, where one HTML page is loaded and JavaScript updates the content dynamically. React apps are SPAs, which makes navigation fast and smooth.

Important Point:

- SPAs need client-side routing like React Router.
- Next.js adds server-side rendering to improve SEO.

Q29. What is Virtual DOM?

Easy English:

The Virtual DOM is a lightweight JavaScript copy of the real DOM.

- When state changes, React builds a new Virtual DOM tree.
- It compares the new tree with the old one (diffing).
- Only the changed parts are updated in the real DOM.

Hinglish:

Virtual DOM asli DOM ki ek halki JavaScript copy hai. State badalne par React nayi copy banata hai, purani se compare karta hai aur sirf badla hua hissa asli DOM me update karta hai. Isse app fast chalti hai.

Example:

```
// We only change state
setCount(count + 1);
// React figures out the minimum DOM update by itself
```

Why is it used?

It is used because direct DOM updates are slow, and updating only the changed nodes improves performance a lot.

Interview Answer:

The Virtual DOM is a lightweight copy of the real DOM kept in memory. React compares the new and old virtual trees and updates only the changed parts of the real DOM, which makes rendering efficient.

Important Point:

- The comparison process is called diffing.
- Virtual DOM is fast, but avoiding unnecessary renders is still important.

Q30. What is reconciliation?

Easy English:

Reconciliation is the process React uses to update the UI efficiently.

- React compares the new Virtual DOM with the previous one.
- It finds the minimum set of changes needed.
- It then updates only those nodes in the real DOM.
- Keys help React match elements correctly in lists.

Hinglish:

Reconciliation wo process hai jisme React nayi Virtual DOM ko purani se compare karta hai, sabse kam changes nikalta hai aur sirf wahi asli DOM me update karta hai. List me keys se React sahi item pehchaan paata hai.

Example:

```
// Only the changed <li> is updated in the real DOM
{todos.map(t => <li key={t._id}>{t.text}</li>)}
```

Why is it used?

It is used so that React can keep the UI in sync with the data without re-rendering the entire page.

Interview Answer:

Reconciliation is React's process of comparing the new Virtual DOM with the previous one and applying only the

PART 5

MONGODB + MONGOOSE

PART 5 - MONGODB + MONGOOSE

MongoDB Basics (Q1 - Q12)

Q1. What is MongoDB?

Easy English:

MongoDB is an open source NoSQL document database.

- It stores data in flexible, JSON-like documents instead of tables and rows.
- Documents are grouped into collections.
- It has no fixed schema, so different documents can have different fields.
- It is the "M" in the MERN stack.

Hinglish:

MongoDB ek NoSQL document database hai. Isme data table aur row me nahi, balki JSON jaise documents me store hota hai. Har document ka structure alag ho sakta hai, isliye ise flexible database kehte hain.

Example:

```
// one document in the "users" collection
{
  "_id": ObjectId("65f1a2b3c4d5e6f7a8b9c0d1"),
  "name": "Deepak",
  "email": "deepak@mail.com",
  "skills": ["React", "Node"]
}
```

Why is it used?

It is used for applications where the data structure changes often and where fast development and easy scaling are needed.

Interview Answer:

MongoDB is a NoSQL document database that stores data in flexible JSON-like documents called BSON. It has no fixed schema and works very naturally with JavaScript, which is why it is used in the MERN stack.

Important Point:

- MongoDB stores data internally as BSON, not plain JSON.
- It scales horizontally using sharding.

Q2. Why do we use MongoDB?

Easy English:

We use MongoDB because:

- Its documents look like JavaScript objects, so it fits perfectly with Node and React.
- The schema is flexible, so we can add fields without changing the whole database.
- It scales horizontally, which handles large amounts of data.
- MongoDB Atlas gives a free managed cloud database.

Hinglish:

MongoDB isliye use karte hain kyunki iska data JavaScript object jaisa lagta hai, schema flexible hota hai, aur scaling aasan hai. Atlas par free cloud database bhi mil jaata hai, isliye MERN projects me bahut popular hai.

Example:

```
const user = await User.create({
  name: "Deepak",
  email: "d@mail.com"
});
```

Why is it used?

It is used for real-time apps, content systems, e-commerce catalogs and any project where requirements keep changing.

Interview Answer:

We use MongoDB because it stores data as JSON-like documents that match JavaScript objects, it has a flexible schema, and it scales easily. This makes development in the MERN stack fast and simple.

Important Point:

- MongoDB is not ideal when there are many complex relations and heavy joins.
- Transactions are supported but SQL is still stronger for them.

Q3. SQL vs NoSQL.

Easy English:

SQL databases store data in tables with rows and columns and need a fixed schema. NoSQL databases like MongoDB store data in documents with a flexible structure.

- SQL uses JOIN for relations, MongoDB uses embedding or $lookup.
- SQL scales vertically, NoSQL scales horizontally.

Hinglish:

SQL database table aur row me data rakhta hai aur schema fix hota hai. NoSQL jaise MongoDB document me data rakhta hai aur structure flexible hota hai. SQL me JOIN hoti hai, MongoDB me embedding ya $lookup.

Example:

```
-- SQL
SELECT * FROM users WHERE city = 'Gorakhpur';

// MongoDB
db.users.find({ city: "Gorakhpur" })
```

Why is it used?

We compare them to choose the right database: SQL for strict structured data like banking, NoSQL for flexible fast-changing data.

Interview Answer:

SQL databases are relational with tables, rows and a fixed schema, while NoSQL databases like MongoDB store flexible JSON-like documents. SQL is better for complex relations and NoSQL is better for flexible, rapidly changing data.

Important Point:

- MongoDB supports ACID transactions since version 4.0.
- Choose SQL when data relations are strict, MongoDB when the schema evolves.

Point	SQL	NoSQL (MongoDB)
Storage	Tables, rows	Collections, documents
Schema	Fixed	Flexible
Relations	JOIN	Embedding or $lookup
Scaling	Vertical	Horizontal (sharding)
Examples	MySQL, PostgreSQL	MongoDB, Cassandra

Q4. What is a database?

Easy English:

A database is an organised place where data is stored so it can be accessed and managed easily.

- In MongoDB, a database contains many collections.
- One project usually uses one database.

Hinglish:

Database wo jagah hai jahan data organised tarike se rakha jaata hai. MongoDB me ek database ke andar bahut saari collections hoti hain. Ek project me aam taur par ek hi database hota hai.

Example:

```
use shopdb                 // switch or create a database
show dbs                     // list all databases
```

Why is it used?

It is used to store application data permanently instead of keeping it in memory where it would be lost.

Interview Answer:

A database is an organised collection of data. In MongoDB, a database holds multiple collections, and each collection holds documents.

Important Point:

- MongoDB creates a database only when the first document is inserted.
- Database name is part of the connection string.

Q5. What is a collection?

Easy English:

A collection is a group of related documents in MongoDB.

- It is similar to a table in SQL, but without a fixed schema.
- Documents in one collection can have different fields.
- Mongoose creates a collection from the model name in lowercase plural form.

Hinglish:

Collection related documents ka group hai, jaise SQL ka table. Fark itna hai ki isme schema fix nahi hota. Mongoose model ka naam lowercase plural banakar collection bana deta hai.

Example:

```
// model "User"  -> collection "users"
db.users.find()
db.createCollection("orders")
```

Why is it used?

It is used to keep similar data together, such as all users in one collection and all products in another.

Interview Answer:

A collection in MongoDB is a group of documents, similar to a table in SQL but schema-less. Mongoose automatically creates a lowercase plural collection from the model name.

Important Point:

- Collections are created automatically on the first insert.
- Keep one type of entity per collection.

Q6. What is a document?

Easy English:

A document is a single record in MongoDB, written in a JSON-like format.

- It is made of field and value pairs.
- Every document has a unique _id field.
- It can hold nested objects and arrays.
- The maximum size of one document is 16 MB.

Hinglish:

Document MongoDB ka ek record hai jo JSON jaisa dikhta hai. Isme field aur value hote hain aur har document me ek unique _id hoti hai. Isme nested object aur array bhi rakh sakte hain.

Example:

```
{
  "_id": ObjectId("..."),
  "name": "Laptop",
  "price": 50000,
  "tags": ["electronics", "computer"],
  "seller": { "name": "ABC", "city": "Kanpur" }
}
```

Why is it used?

It is used to store all information about one entity in one place, which reduces the need for joins.

Interview Answer:

A document is a single record in MongoDB stored in BSON format with field-value pairs. Every document has a unique _id, and it can contain nested objects and arrays.

Important Point:

- Maximum document size is 16 MB.
- Documents are stored as BSON, which supports more data types than JSON.

Q7. What is BSON?

Easy English:

BSON means Binary JSON. It is the format MongoDB uses internally to store data.

- It is a binary version of JSON, so it is faster to read and write.
- It supports extra data types like ObjectId, Date, Decimal128 and Binary.

Hinglish:

BSON ka matlab Binary JSON hai. MongoDB data ko isi format me store karta hai. Ye binary hone ki wajah se fast hota hai aur isme ObjectId aur Date jaise extra data types bhi support hote hain.

Example:

```
// We write JSON
{ "createdAt": "2026-01-01" }

// MongoDB stores BSON with real types
{ createdAt: ISODate("2026-01-01T00:00:00Z") }
```

Why is it used?

It is used because plain JSON cannot store types like dates and binary data efficiently.

Interview Answer:

BSON stands for Binary JSON and is the storage format used by MongoDB. It is faster to parse than JSON and supports extra types like ObjectId, Date and Binary.

Important Point:

- JSON is for humans, BSON is for the machine.
- BSON is why MongoDB can store real Date objects.

Q8. What is _id?

Easy English:

_id is the primary key of every MongoDB document.

- It must be unique inside a collection.
- If we do not provide it, MongoDB creates an ObjectId automatically.
- It is indexed by default, so searching by _id is very fast.

Hinglish:

_id har MongoDB document ki primary key hai aur collection me unique hoti hai. Agar hum na dein to MongoDB khud ObjectId bana deta hai. Ye by default indexed hoti hai isliye search fast hota hai.

Example:

```
const user = await User.findById(req.params.id);
db.users.findOne({ _id: ObjectId("65f1a2b3c4d5e6f7a8b9c0d1") })
```

Why is it used?

It is used to identify, fetch, update and delete a specific document.

Interview Answer:

_id is the unique primary key of every MongoDB document. MongoDB generates it automatically as an ObjectId, and it is indexed by default for fast lookups.

Important Point:

- _id cannot be changed after the document is created.
- You can set a custom _id, for example an email, if it is unique.

Q9. What is ObjectId?

Easy English:

ObjectId is the default type of _id in MongoDB.

- It is a 12-byte unique value shown as a 24 character hexadecimal string.
- It contains a timestamp, so documents are roughly sorted by creation time.
- We can extract the creation time from it.

Hinglish:

ObjectId MongoDB ki default _id type hai. Ye 12 byte ki unique value hai jo 24 characters ki string me dikhti hai. Isme timestamp hota hai, isliye isse creation time bhi nikal sakte hain.

Example:

```
const id = new mongoose.Types.ObjectId();
console.log(id.getTimestamp());

// validate before using
mongoose.Types.ObjectId.isValid(req.params.id);
```

Why is it used?

It is used to create unique ids without a central counter, which is important for distributed databases.

Interview Answer:

ObjectId is MongoDB's default id type, a 12-byte unique value displayed as a 24 character hex string. It includes a timestamp, so it is unique and roughly sortable by creation time.

Important Point:

- Always validate an ObjectId from the URL, otherwise Mongoose throws a CastError.
- ObjectId is not the same as a plain string, comparison needs care.

Q10. What is MongoDB Atlas?

Easy English:

MongoDB Atlas is the official cloud service of MongoDB.

- The database is hosted and managed by MongoDB itself.
- It handles backups, security, updates and scaling.
- It has a free tier which is enough for student projects.
- We connect using a connection string (mongodb+srv://...).

Hinglish:

MongoDB Atlas MongoDB ki official cloud service hai. Isme database cloud par hota hai aur backup, security, scaling sab MongoDB khud sambhalta hai. Free tier student projects ke liye kaafi hai.

Example:

```
MONGO_URI=mongodb+srv://user:pass@cluster0.mongodb.net/shopdb
```

Why is it used?

It is used so that a deployed app can access the database from anywhere, without installing MongoDB on a server.

Interview Answer:

MongoDB Atlas is MongoDB's managed cloud database service. It handles hosting, backups and scaling, and we connect to it using a connection string stored in the .env file.

Important Point:

- You must whitelist the IP address in Atlas network access.
- Never commit the Atlas connection string to GitHub.

Q11. What is MongoDB Compass?

Easy English:

MongoDB Compass is the official GUI (graphical tool) for MongoDB.

- It lets us view databases, collections and documents visually.
- We can run queries, edit documents and create indexes without the shell.
- It also shows query performance.

Hinglish:

MongoDB Compass MongoDB ka official GUI tool hai. Isme database aur documents aankhon se dekh sakte hain, query chala sakte hain aur document edit kar sakte hain, wo bhi bina command likhe.

Example:

```
// Paste the same connection string used in code
mongodb+srv://user:pass@cluster0.mongodb.net/
```

Why is it used?

It is used during development to quickly check whether the data saved by the API is correct.

Interview Answer:

MongoDB Compass is the official GUI client for MongoDB. It allows us to browse collections, run queries and edit documents visually, which is very helpful during development and debugging.

Important Point:

- Compass can analyse a schema and suggest indexes.
- Atlas also provides a similar browser-based data explorer.

Q12. What is CRUD?

Easy English:

CRUD stands for Create, Read, Update and Delete, the four basic database operations.

- Create: insertOne, insertMany, Model.create
- Read: find, findOne, findById
- Update: updateOne, updateMany, findByIdAndUpdate
- Delete: deleteOne, deleteMany, findByIdAndDelete

Hinglish:

CRUD ka matlab Create, Read, Update aur Delete hai. Ye database ke chaar basic kaam hain. Har REST API inhi chaar operations par bani hoti hai.

Example:

```
await Product.create({ name: "Pen", price: 10 });   // C
await Product.find();                                 // R
await Product.findByIdAndUpdate(id, { price: 12 });   // U
await Product.findByIdAndDelete(id);                  // D
```

Why is it used?

It is used as the base of every backend application, because almost every feature is some form of CRUD.

Interview Answer:

CRUD means Create, Read, Update and Delete. These are the four basic operations of any database, and in a MERN project they map to POST, GET, PUT/PATCH and DELETE routes.

Important Point:

- CRUD maps directly to REST API methods.
- Always validate input before performing a create or update.

CRUD Operations and Queries (Q13 - Q27)

Q13. What is insertOne()?

Easy English:

insertOne() adds a single document to a collection.

- If _id is not given, MongoDB creates one.
- It returns the inserted id and an acknowledgement.
- In Mongoose we normally use Model.create() instead.

Hinglish:

insertOne() ek document collection me daalta hai. _id na dene par MongoDB khud bana deta hai. Mongoose me iski jagah Model.create() use karte hain.

Example:

```
// Mongo shell
db.users.insertOne({ name: "Deepak", city: "Gorakhpur" });

// Mongoose
const user = await User.create({ name: "Deepak" });
```

Why is it used?

It is used whenever one new record has to be saved, for example one new user or one new order.

Interview Answer:

insertOne() inserts a single document into a collection and returns the inserted id. In Mongoose we usually achieve the same with Model.create() or new Model().save().

Important Point:

- Mongoose applies schema validation, the raw shell does not.
- The collection is created automatically on the first insert.

Q14. What is insertMany()?

Easy English:

insertMany() inserts several documents at once.

- It takes an array of documents.
- It is much faster than calling insertOne in a loop.
- By default it stops on the first error (ordered: true).

Hinglish:

insertMany() ek saath kai documents daalta hai. Isme array pass karte hain. Loop me insertOne chalane se ye kaafi fast hota hai.

Example:

```
await Product.insertMany([
  { name: "Pen", price: 10 },
  { name: "Book", price: 100 }
]);
```

Why is it used?

It is used for seeding sample data, bulk imports and migrating data.

Interview Answer:

insertMany() inserts an array of documents in a single operation, which is faster than inserting them one by one. It is commonly used for seeding and bulk data imports.

Important Point:

- Use ordered: false to continue inserting even if one document fails.
- insertMany skips some Mongoose middleware by default.

Q15. What is find()?

Easy English:

find() returns all documents matching a filter.

- An empty filter {} returns everything in the collection.
- It returns an array (a cursor in the shell).
- We can add projection, sort, skip and limit.

Hinglish:

find() filter se match karne wale saare documents laata hai. Khaali {} dene par poori collection aa jaati hai. Isme sort, limit aur skip bhi laga sakte hain.

Example:

```
const products = await Product.find({ category: "mobile" })
  .select("name price")
  .sort({ price: -1 })
  .limit(10);
```

Why is it used?

It is used for listing data such as all products, all posts or filtered search results.

Interview Answer:

find() returns all documents that match the given filter as an array. We can chain select, sort, skip and limit to control the result, which is how pagination is built.

Important Point:

- find returns an array, findOne returns a single object or null.
- Always use limit on large collections.

Q16. What is findOne()?

Easy English:

findOne() returns the first document matching the filter.

- If nothing matches, it returns null.
- It is used when we expect only one record.

Hinglish:

findOne() filter se match hone wala pehla document laata hai. Kuch na mile to null aata hai. Jab ek hi record chahiye, tab yahi use karte hain.

Example:

```
const user = await User.findOne({ email: "d@mail.com" });
if (!user) return res.status(404).json({ message: "Not found" });
```

Why is it used?

It is used in login, checking for a duplicate email and fetching a record by a unique field.

Interview Answer:

findOne() returns the first matching document or null if none matches. It is used when we expect a single record, for example finding a user by email during login.

Important Point:

- Always check for null before using the result.
- findById is a shortcut for findOne({ _id: id }).

Q17. What is updateOne()?

Easy English:

updateOne() updates the first document that matches the filter.

- It needs update operators like $set or $inc.
- It returns information about how many documents matched and were modified.
- It does not return the document itself.

Hinglish:

updateOne() sirf pehla matching document update karta hai. Isme $set jaise operators lagte hain. Ye document return nahi karta, sirf batata hai kitne update hue.

Example:

```
await User.updateOne(
  { email: "d@mail.com" },
  { $set: { city: "Kanpur" } }
);
```

Why is it used?

It is used to change one specific record, for example updating a profile or a status.

Interview Answer:

updateOne() updates the first document matching a filter using operators like $set. It returns the update result, not the document, so we use findByIdAndUpdate when we need the updated data back.

Important Point:

- Without $set, some drivers replace the whole document.
- Use upsert: true to insert if no match is found.

Q18. What is updateMany()?

Easy English:

updateMany() updates all documents that match the filter.

- It uses the same update operators as updateOne.
- It is used for bulk updates.

Hinglish:

updateMany() saare matching documents ko ek saath update kar deta hai. Operators wahi hote hain jo updateOne me lagte hain. Bulk change ke liye ye best hai.

Example:

```
await Product.updateMany(
  { category: "mobile" },
  { $set: { onSale: true } }
);
```

Why is it used?

It is used for operations like applying a discount to a category or marking all old orders as archived.

Interview Answer:

updateMany() updates every document that matches the filter in one operation. It is used for bulk changes like applying a discount to a whole category.

Important Point:

- An empty filter {} updates the entire collection, so be careful.
- Test the filter with find() before running updateMany.

Q19. What is deleteOne()?

Easy English:

deleteOne() removes the first document matching the filter.

- It returns deletedCount.
- In Mongoose, findByIdAndDelete also returns the deleted document.

Hinglish:

deleteOne() pehla matching document delete karta hai aur batata hai kitne delete hue. Mongoose me findByIdAndDelete deleted document bhi return kar deta hai.

Example:

```
await User.deleteOne({ _id: req.params.id });

const removed = await User.findByIdAndDelete(req.params.id);
```

Why is it used?

It is used to delete a single record such as one post, one cart item or one account.

Interview Answer:

deleteOne() deletes the first document that matches the filter and returns the deleted count. In Mongoose, findByIdAndDelete is often preferred because it also returns the removed document.

Important Point:

- Deletion is permanent, so protect these routes.
- Many projects use soft delete with an isDeleted flag instead.

Q20. What is deleteMany()?

Easy English:

deleteMany() removes all documents matching the filter.

- It returns how many documents were deleted.
- An empty filter deletes the entire collection's documents.

Hinglish:

deleteMany() saare matching documents hata deta hai aur count return karta hai. Khaali filter dene par poori collection khaali ho jaayegi, isliye dhyaan se use karein.

Example:

```
await Cart.deleteMany({ userId: req.user.id });

// dangerous
// await Cart.deleteMany({});
```

Why is it used?

It is used for cleanup tasks like clearing a cart, removing expired tokens or deleting old logs.

Interview Answer:

deleteMany() deletes all documents matching a filter and returns the deleted count. It is used for bulk cleanup, and an empty filter would delete everything, so it must be used carefully.

Important Point:

- Always double check the filter before running it.
- Use TTL indexes to auto-delete expired documents.

Q21. What is MongoDB query?

Easy English:

A MongoDB query is an object that describes which documents we want.

- Keys are field names and values are the conditions.
- Multiple keys act as AND by default.
- Operators start with a dollar sign, like $gt and $in.

Hinglish:

MongoDB query ek object hota hai jo batata hai kaunse documents chahiye. Field aur uski condition likhte hain. Do conditions likhne par by default AND lagta hai, aur operators dollar sign se shuru hote hain.

Example:

```
// price > 1000 AND category is mobile
db.products.find({ price: { $gt: 1000 }, category: "mobile" });

// name contains "pen" (case-insensitive)
db.products.find({ name: { $regex: "pen", $options: "i" } });
```

Why is it used?

It is used to fetch exactly the data needed, which keeps the API fast and the response small.

Interview Answer:

A MongoDB query is a JSON-like object that describes the filter conditions. Field names map to values or operators like $gt and $in, and multiple fields are combined with AND by default.

Important Point:

- Never pass raw user input directly into a query, it can cause NoSQL injection.
- Use projection to return only needed fields.

Q22. What are comparison operators?

Easy English:

Comparison operators compare a field value with a given value.

- $eq equal, $ne not equal
- $gt greater than, $gte greater than or equal
- $lt less than, $lte less than or equal
- $in value is in a list, $nin value is not in a list

Hinglish:

Comparison operators field ki value compare karte hain. $gt bada, $lt chhota, $gte bada ya barabar, $in list me se koi ek, aur $ne barabar nahi. Ye filtering me bahut kaam aate hain.

Example:

```
// price between 1000 and 5000
db.products.find({ price: { $gte: 1000, $lte: 5000 } });

// category is mobile or laptop
db.products.find({ category: { $in: ["mobile", "laptop"] } });
```

Why is it used?

They are used for filtering, price ranges, date ranges and search features in an API.

Interview Answer:

Comparison operators such as $gt, $lt, $gte, $lte, $ne and $in are used to compare field values in a MongoDB query. They are the basis of filtering and range searches.

Important Point:

- $in takes an array of possible values.

- Comparison operators work on dates as well as numbers.

Q23. What are logical operators?

Easy English:

Logical operators combine multiple conditions.

- $and: all conditions must be true
- $or: at least one condition must be true
- $nor: none of the conditions should be true
- $not: reverses a condition

Hinglish:

Logical operators kai conditions ko jodte hain. $and me sab sach hona chahiye, $or me koi ek, $nor me koi bhi nahi, aur $not condition ko ulta kar deta hai.

Example:

```
db.products.find({
  $or: [ { category: "mobile" }, { price: { $lt: 500 } } ]
});

db.users.find({ $and: [ { active: true }, { age: { $gte: 18 } } ] });
```

Why is it used?

They are used to build advanced search filters, such as searching in the name or the description field.

Interview Answer:

Logical operators like $and, $or, $nor and $not combine multiple conditions in a query. Writing several fields in one object is an implicit $and, while $or is needed for either-or conditions.

Important Point:

- Multiple fields in one object are already an AND.
- $or can be slow without proper indexes.

Q24. What is $set?

Easy English:

$set is an update operator that sets or changes the value of specific fields.

- Fields not mentioned stay unchanged.
- If the field does not exist, it is created.

Hinglish:

$set update operator hai jo sirf bataye gaye fields ki value badalta hai. Baaki fields waise hi rehte hain, aur field na ho to naya ban jaata hai.

Example:

```
await User.updateOne(
  { _id: id },
  { $set: { city: "Kanpur", "address.pin": "208001" } }
);
```

Why is it used?

It is used for partial updates such as changing a status, price or profile field.

Interview Answer:

$set is the MongoDB update operator used to modify specific fields without affecting the rest of the document. It is the operator behind partial updates like PATCH.

Important Point:

- Without $set, an update can replace the entire document.
- Dot notation lets us update nested fields.

Q25. What is $push?

Easy English:

$push adds a new value to the end of an array field.

- It allows duplicate values.
- With $each we can push multiple values at once.
- $addToSet pushes only if the value is not already present.

Hinglish:

$push array field me nayi value add karta hai. Ye duplicate bhi allow karta hai. Ek saath kai value daalni ho to $each use karte hain, aur duplicate nahi chahiye to $addToSet.

Example:

```
await Post.updateOne(
  { _id: postId },
  { $push: { comments: { user: userId, text: "Nice post" } } }
);
```

Why is it used?

It is used to add items to arrays such as comments, cart items, tags and order history.

Interview Answer:

$push appends a value to an array field in a document. It allows duplicates, while $addToSet adds a value only if it does not already exist.

Important Point:

- $each pushes multiple values in one operation.
- Avoid unbounded arrays, the 16 MB document limit applies.

Q26. What is $pull?

Easy English:

$pull removes all values from an array that match a condition.

- It is the opposite of $push.
- The condition can be a simple value or a query object.

Hinglish:

$pull array me se matching value hata deta hai. Ye $push ka ulta hai. Condition simple value bhi ho sakti hai aur object bhi.

Example:

```
await Cart.updateOne(
  { userId },
  { $pull: { items: { productId: pid } } }
);
```

Why is it used?

It is used to remove a cart item, unlike a post, remove a tag or delete a comment from an array.

Interview Answer:

$pull removes elements from an array that match a given condition. It is the opposite of $push and is used for actions like removing an item from a cart.

Important Point:

- $pop removes from the start or end of an array.
- $pull removes all matching elements, not just the first one.

Q27. What is indexing?

Easy English:

An index is a special data structure that makes searching faster.

- Without an index, MongoDB scans every document (a collection scan).
- With an index, it jumps directly to the matching data.
- _id is indexed automatically.
- Indexes can be single field, compound, text or unique.

Hinglish:

Index ek special structure hai jo search fast banata hai, jaise kitaab ka index page. Bina index ke MongoDB har document check karta hai. _id par index by default hota hai.

Example:

```
// create an index
db.users.createIndex({ email: 1 }, { unique: true });

// Mongoose schema
email: { type: String, required: true, unique: true, index: true }
```

Why is it used?

It is used to keep queries fast when a collection grows to thousands or millions of documents.

Interview Answer:

An index is a data structure that speeds up queries by avoiding a full collection scan. MongoDB indexes _id by default, and we add indexes on fields we search or sort by frequently.

Important Point:

- Indexes speed up reads but slow down writes and use extra storage.
- Use explain() to check if a query is using an index.

Q28. Why do we use indexes?

Easy English:

We use indexes because:

- They make find, sort and filter operations much faster.
- They avoid scanning the whole collection.
- A unique index prevents duplicate values like duplicate emails.

Hinglish:

Index isliye use karte hain kyunki search aur sorting bahut fast ho jaati hai aur poori collection scan nahi karni padti. Unique index duplicate email jaisi problem bhi rok deta hai.

Example:

```
// fast search on email
db.users.createIndex({ email: 1 }, { unique: true });

// compound index for filter + sort
db.products.createIndex({ category: 1, price: -1 });
```

Why is it used?

They are used on fields used in search, filters, sorting and joins, such as email, category and createdAt.

Interview Answer:

We use indexes to make queries fast and to avoid full collection scans. A unique index also enforces data integrity, for example preventing two users with the same email.

Important Point:

- Do not index every field, it slows down inserts and updates.

- Compound index order matters, it follows the query pattern.

Q29. What is aggregation?

Easy English:

Aggregation is the process of processing data and returning computed results.

- It can group, filter, sort, count and calculate totals or averages.
- It is similar to GROUP BY in SQL, but more powerful.
- It is done with the aggregation pipeline.

Hinglish:

Aggregation ka matlab hai data ko process karke result nikalna, jaise total, average ya count. Ye SQL ke GROUP BY jaisa hai lekin usse zyada powerful, aur pipeline ke through kaam karta hai.

Example:

```
const result = await Order.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$userId", total: { $sum: "$amount" } } }
]);
```

Why is it used?

It is used for reports and dashboards, like total sales per month or the number of orders per user.

Interview Answer:

Aggregation is MongoDB's way of processing documents and returning computed results such as totals, averages and grouped data. It works through a pipeline of stages like $match, $group and $sort.

Important Point:

- Always put $match first to reduce the number of documents early.
- Aggregation runs on the server, which is faster than doing it in Node.

Q30. What is aggregation pipeline?

Easy English:

The aggregation pipeline is an array of stages where the output of one stage becomes the input of the next.

- Common stages: $match, $group, $sort, $project, $limit, $skip, $lookup, $unwind.
- Documents flow through the stages one after another.

Hinglish:

Aggregation pipeline stages ka array hai jisme ek stage ka output agle stage ka input banta hai. Common stages hain $match, $group, $sort, $project aur $lookup. Data ek ke baad ek stage se guzarta hai.

Example:

```
await Order.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$city", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } },
  { $limit: 5 }
]);
```

Why is it used?

It is used to build complex reports in a single database query instead of fetching all data into Node.js.

Interview Answer:

The aggregation pipeline is a sequence of stages where each stage transforms the documents and passes them to the next. Stages like $match, $group and $sort let us build reports in one query.

Important Point:

- Stage order strongly affects performance.
- $project controls which fields appear in the output.

Q31. What is $match?

Easy English:

$match filters documents in the aggregation pipeline.

- It works like the filter in find().
- It should be placed as early as possible to reduce the data.
- It can use indexes when it is the first stage.

Hinglish:

$match aggregation me documents filter karta hai, bilkul find() ke filter jaisa. Ise pipeline me sabse pehle rakhna chahiye taaki aage kam data jaaye aur query fast rahe.

Example:

```
{ $match: { status: "paid", amount: { $gte: 500 } } }
```

Why is it used?

It is used to select only the relevant documents before grouping or calculating.

Interview Answer:

$match filters documents in an aggregation pipeline, similar to the find filter. Placing it first improves performance because it reduces the documents passed to later stages.

Important Point:

- Only the first $match can use an index.
- It supports all normal query operators.

Q32. What is $group?

Easy English:

$group groups documents by a field and applies accumulator operators.

- The _id field defines what to group by.
- Accumulators: $sum, $avg, $min, $max, $count, $push.

Hinglish:

$group documents ko kisi field ke hisaab se group karta hai. _id batata hai kis field par group karna hai, aur $sum ya $avg jaise operators calculation karte hain.

Example:

```
{ $group: {
    _id: "$category",
    totalSales: { $sum: "$amount" },
    avgPrice: { $avg: "$price" },
    count: { $sum: 1 }
} }
```

Why is it used?

It is used for reports like sales per category, orders per user or average rating per product.

Interview Answer:

$group groups documents by a key defined in _id and computes values using accumulators like $sum and $avg. It is the MongoDB equivalent of SQL GROUP BY.

Important Point:

- Use _id: null to group the entire collection into one result.
- Field names inside $group must be written with a dollar prefix.

Q33. What is $sort?

Easy English:

$sort orders the documents in the pipeline.

- 1 means ascending, -1 means descending.
- It can sort on multiple fields.

Hinglish:

$sort documents ko order me lagata hai. 1 dene par chhote se bade, aur -1 dene par bade se chhote. Ek se zyada field par bhi sort kar sakte hain.

Example:

```
{ $sort: { totalSales: -1, name: 1 } }

// in a normal query
await Product.find().sort({ createdAt: -1 });
```

Why is it used?

It is used to show top results, newest items first, or price low to high.

Interview Answer:

$sort arranges documents in ascending order with 1 or descending order with -1. It is used for leaderboards, latest items and price sorting.

Important Point:

- Sorting a large result without an index can exceed the memory limit.
- Sorting after $group usually cannot use an index.

Q34. What is $lookup?

Easy English:

$lookup performs a left outer join with another collection.

- It matches a local field with a foreign field.
- The joined documents come back as an array.
- We often use $unwind after it to flatten that array.

Hinglish:

$lookup doosri collection ke saath join karta hai, jaise SQL ki left join. Local field aur foreign field match karke result array me aata hai, jise $unwind se flat kar lete hain.

Example:

```
{ $lookup: {
    from: "users",
    localField: "userId",
    foreignField: "_id",
    as: "user"
} },
{ $unwind: "$user" }
```

Why is it used?

It is used when we need related data from two collections in one query, for example orders with user details.

Interview Answer:

$lookup performs a left outer join between two collections in an aggregation pipeline. The matched documents are returned as an array, and $unwind is often used to flatten it.

Important Point:

- In Mongoose, populate() is a simpler alternative for common cases.
- $lookup can be slow without an index on the foreign field.

Q35. What is embedding?

Easy English:

Embedding means storing related data inside the same document.

- The child data is nested as an object or an array.
- Reading is fast because one query gets everything.

Hinglish:

Embedding ka matlab hai related data ko usi document ke andar rakhna. Isse ek hi query me poora data mil jaata hai aur read fast hota hai.

Example:

```
{
  "_id": 1,
  "name": "Deepak",
  "address": { "city": "Gorakhpur", "pin": "273001" },
  "phones": ["9999999999"]
}
```

Why is it used?

It is used for one-to-few relationships where the child data is always read together with the parent, like an address inside a user.

Interview Answer:

Embedding means storing related data inside the same document as nested objects or arrays. It makes reads fast because one query returns everything, and it suits one-to-few relationships.

Important Point:

- Documents have a 16 MB limit, so do not embed unbounded arrays.
- Duplicated embedded data is harder to keep updated.

Q36. What is referencing?

Easy English:

Referencing means storing only the _id of a related document in another collection.

- The real data lives in a separate collection.
- We join it later using populate() or $lookup.

Hinglish:

Referencing me hum doosre document ki sirf _id store karte hain, asli data alag collection me rehta hai. Baad me populate() ya $lookup se jod lete hain.

Example:

```
// order document
{ "_id": 1, "userId": ObjectId("..."), "amount": 500 }

// join later
await Order.find().populate("userId", "name email");
```

Why is it used?

It is used for one-to-many and many-to-many relationships where the same data is shared by many documents.

Interview Answer:

Referencing means storing the _id of a related document instead of the full data. The relation is resolved later using populate or $lookup, and it avoids data duplication.

Important Point:

- It needs an extra query or a join to fetch the related data.
- It keeps documents small and data consistent.

Q37. Embedding vs referencing.

Easy English:

Embedding keeps related data inside one document, referencing keeps it in a separate collection and stores only the id.

- Embedding: faster reads, possible duplication, document can grow large.
- Referencing: no duplication, easier updates, but needs a join.

Hinglish:

Embedding me data ek hi document ke andar hota hai, isliye read fast hai lekin duplication ka risk hai. Referencing me sirf id rakhi jaati hai, data alag rehta hai, isliye update aasan hai lekin join karna padta hai.

Example:

```
// Embedded (one-to-few)
{ name: "Deepak", address: { city: "GKP" } }

// Referenced (one-to-many)
{ name: "Deepak", orders: [ObjectId("..."), ObjectId("...")] }
```

Why is it used?

Choosing correctly is a key MongoDB design decision that affects performance and data consistency.

Interview Answer:

Embedding stores related data inside the same document and is best for one-to-few data read together. Referencing stores only the id and is best for one-to-many or shared data that changes often.

Important Point:

- Rule of thumb: embed for one-to-few, reference for one-to-many.
- If data is updated often and shared, prefer referencing.

Point	Embedding	Referencing
Storage	Inside same document	Separate collection
Read speed	Fast, one query	Needs join or populate
Duplication	Possible	No
Best for	One-to-few	One-to-many, many-to-many
Risk	16 MB document limit	Extra queries

Mongoose (Q38 - Q50)

Q38. What is Mongoose?

Easy English:

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js.

- It adds schemas and models on top of MongoDB.
- It provides validation, middleware, population and helpful query methods.
- It gives structure to a schema-less database.

Hinglish:

Mongoose ek ODM library hai jo MongoDB aur Node ke beech kaam karti hai. Ye schema aur model deti hai, validation karti hai aur query likhna aasan banati hai. Matlab MongoDB ko structure mil jaata hai.

Example:

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({ name: String });
const User = mongoose.model("User", userSchema);
```

Why is it used?

It is used because raw MongoDB has no validation, so a wrong field can be saved by mistake.

Interview Answer:

Mongoose is an Object Data Modeling library for MongoDB in Node.js. It provides schemas, models, validation and middleware, which adds structure and safety to a schema-less database.

Important Point:

- ODM for MongoDB is like ORM for SQL.
- Mongoose is a library, MongoDB is the database.

Q39. Why do we use Mongoose?

Easy English:

We use Mongoose because:

- Schema validation stops wrong or missing data from being saved.
- Models give clean methods like find, create and findByIdAndUpdate.
- Middleware (hooks) lets us run code before saving, for example hashing a password.
- populate() makes relations easy.
- timestamps add createdAt and updatedAt automatically.

Hinglish:

Mongoose isliye use karte hain kyunki validation se galat data save nahi hota, model se queries clean lagti hain, hooks se password hash kar sakte hain, aur populate se relations aasan ho jaate hain.

Example:

```
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 18 }
}, { timestamps: true });
```

Why is it used?

It is used in almost every MERN project because it makes the backend safer and faster to build.

Interview Answer:

We use Mongoose because it adds schema validation, models, middleware and population on top of MongoDB. This prevents bad data and makes database code much cleaner.

Important Point:

- Validation happens in the application layer, not in the database.
- Direct MongoDB driver updates can bypass Mongoose validation.

Q40. What is Mongoose Schema?

Easy English:

A Schema defines the structure of documents in a collection.

- It lists fields, their types and rules.
- Rules include required, default, unique, min, max and enum.
- Options like timestamps can be added.

Hinglish:

Schema batata hai ki document me kaunse fields honge aur unka type kya hoga. Isme required, default, unique jaise rules bhi likhte hain, jisse galat data save nahi hota.

Example:

```
const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  price: { type: Number, required: true, min: 0 },
  category: { type: String, enum: ["mobile", "laptop"] },
```

```
  inStock: { type: Boolean, default: true }
}, { timestamps: true });
```

Why is it used?

It is used to keep data consistent so that every document has the expected shape.

Interview Answer:

A Mongoose Schema defines the structure and rules of documents in a collection, including field types, required fields, defaults and validations. It brings structure to MongoDB.

Important Point:

- A schema is only a blueprint, the model does the database work.
- Fields not in the schema are ignored by default.

Q41. What is Mongoose Model?

Easy English:

A Model is a class created from a schema that lets us work with the database.

- It provides methods like create, find, findById, updateOne and deleteOne.
- Each document is an instance of the model.
- The collection name is the lowercase plural of the model name.

Hinglish:

Model schema se banta hai aur database se baat karta hai. Isme create, find, update aur delete jaise methods milte hain. Model ka naam "User" ho to collection "users" ban jaati hai.

Example:

```
const User = mongoose.model("User", userSchema);

const user = await User.create({ name: "Deepak" });
const all = await User.find();
```

Why is it used?

It is used to perform all CRUD operations in a clean and validated way.

Interview Answer:

A Mongoose Model is created from a schema and provides the methods used to interact with the database, such as create, find and findByIdAndUpdate. Each document is an instance of the model.

Important Point:

- Schema defines the structure, model performs the operations.
- Model names are singular and capitalised by convention.

Q42. What is validation in Mongoose?

Easy English:

Validation means checking data before saving it to the database.

- Built-in validators: required, min, max, minlength, maxlength, enum, match.
- We can also write custom validators.
- Validation runs on save and create, and on update only if runValidators is set.

Hinglish:

Validation ka matlab hai data save karne se pehle check karna. required, min, max, enum jaise built-in validators hote hain aur custom bhi likh sakte hain. Update par validation chalane ke liye runValidators true karna padta hai.

Example:

```
email: {
  type: String,
  required: [true, "Email is required"],
  match: [/.+@.+\..+/, "Invalid email"]
```

```
}

await User.findByIdAndUpdate(id, data, { runValidators: true });
```

Why is it used?

It is used to stop invalid data from entering the database, which keeps the application reliable.

Interview Answer:

Validation in Mongoose checks data against schema rules before saving. It includes built-in validators like required, min, enum and match, and validation errors can be caught and returned to the client.

Important Point:

- Update queries skip validation unless runValidators: true is passed.
- Validation errors have the name ValidationError.

Q43. What are Mongoose middleware/hooks?

Easy English:

Middleware, also called hooks, are functions that run before or after certain operations.

- pre hooks run before an action, post hooks run after.
- Common uses: hashing a password before save, logging, cleaning related data after delete.

Hinglish:

Mongoose middleware ya hooks wo functions hain jo kisi operation se pehle (pre) ya baad me (post) chalte hain. Sabse common use hai save se pehle password hash karna.

Example:

```
userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

Why is it used?

They are used to keep repeated logic in the model instead of writing it in every controller.

Interview Answer:

Mongoose middleware, or hooks, are functions that run before or after operations like save, find or remove. A common example is a pre-save hook that hashes the password before storing it.

Important Point:

- Do not use an arrow function in a hook, because this will be wrong.
- findByIdAndUpdate does not trigger the save hook.

Q44. What is populate()?

Easy English:

populate() replaces a stored ObjectId with the actual document from the referenced collection.

- It works on fields defined with ref in the schema.
- We can select only the fields we need.
- It performs an extra query behind the scenes.

Hinglish:

populate() stored ObjectId ki jagah asli document le aata hai. Iske liye schema me ref likhna padta hai. Ye andar se ek extra query chalata hai, isliye zarurat ke fields hi select karein.

Example:

```
const orderSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User" }
});
```

```
const orders = await Order.find().populate("user", "name email");
```

Why is it used?

It is used to show related data, for example a post with its author's name or an order with customer details.

Interview Answer:

populate() replaces a referenced ObjectId with the actual document from the other collection. It requires a ref in the schema and is Mongoose's simpler alternative to $lookup.

Important Point:

- populate runs an extra query, so avoid deep nested populates.
- Always select only the fields needed, never the password.

Q45. What is lean()?

Easy English:

lean() makes a query return plain JavaScript objects instead of full Mongoose documents.

- The result is lighter and faster.
- But the objects have no Mongoose methods like save() or virtuals.

Hinglish:

lean() query ka result plain JavaScript object me deta hai, Mongoose document me nahi. Isse response fast aur halka ho jaata hai, lekin save() jaise methods nahi milte.

Example:

```
const products = await Product.find().lean();
// products are plain objects, faster for read-only APIs
```

Why is it used?

It is used in read-only APIs like listing pages, where we only send data to the frontend and do not modify documents.

Interview Answer:

lean() returns plain JavaScript objects instead of Mongoose documents, which makes read queries faster and use less memory. It is used for read-only endpoints where we do not need document methods.

Important Point:

- Do not use lean() if you plan to call save() on the result.
- Virtuals and getters do not work with lean.

Q46. What is timestamps in Mongoose?

Easy English:

timestamps is a schema option that automatically adds two date fields.

- createdAt: when the document was created.
- updatedAt: when it was last updated.
- Mongoose maintains these fields by itself.

Hinglish:

timestamps schema ka ek option hai jo do fields khud add kar deta hai - createdAt aur updatedAt. Inhe Mongoose khud manage karta hai, humein manually set nahi karna padta.

Example:

```
const postSchema = new mongoose.Schema({
  title: String
}, { timestamps: true });

await Post.find().sort({ createdAt: -1 });   // newest first
```

Why is it used?

It is used to sort by newest, show "posted 2 days ago" and to track when records change.

Interview Answer:

The timestamps option in a Mongoose schema automatically adds and maintains createdAt and updatedAt fields. It is commonly used to sort records by newest first.

Important Point:

- You can rename the fields in the timestamps option.
- Index createdAt if you sort by it often.

Q47. What is MongoDB connection?

Easy English:

A MongoDB connection is the link between our Node.js application and the database.

- It is created using a connection string (URI).
- The URI contains the username, password, host and database name.
- The connection should be created once when the server starts.

Hinglish:

MongoDB connection hamare Node app aur database ke beech ka rishta hai. Ye connection string se banta hai jisme username, password, host aur database ka naam hota hai. Ise server start hote waqt ek hi baar banana chahiye.

Example:

```
MONGO_URI=mongodb+srv://user:pass@cluster0.mongodb.net/shopdb

// local
// mongodb://127.0.0.1:27017/shopdb
```

Why is it used?

Without a connection, the application cannot read or write any data.

Interview Answer:

A MongoDB connection links the Node.js application to the database using a connection string that contains credentials, host and database name. It is established once at server startup.

Important Point:

- Keep the URI in .env, never in the code.
- Mongoose maintains a connection pool, so do not connect on every request.

Q48. How do we connect MongoDB with Node.js?

Easy English:

We connect using mongoose.connect() with the URI from process.env.

- We write it in an async function with try...catch.
- We call it before starting the server.
- On failure we log the error and exit the process.

Hinglish:

Connect karne ke liye mongoose.connect() use karte hain aur URI .env se lete hain. Ise async function me try...catch ke saath likhte hain aur server start karne se pehle call karte hain.

Example:

```
// config/db.js
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected");
  } catch (err) {
```

```
    console.error(err.message);
    process.exit(1);
  }
};
module.exports = connectDB;

// server.js
connectDB();
app.listen(5000);
```

Why is it used?

It is used as the standard setup step of every MERN backend project.

Interview Answer:

We connect MongoDB to Node using mongoose.connect with the URI from process.env inside an async function with try...catch. We call it before app.listen so the server starts with a working database connection.

Important Point:

- Call dotenv.config() before using process.env.MONGO_URI.
- process.exit(1) stops the server if the database is unreachable.

Q49. MongoDB Atlas vs local MongoDB.

Easy English:

Local MongoDB runs on our own computer, Atlas runs in the cloud and is managed by MongoDB.

- Local: fast, works offline, but only on that machine and we manage backups ourselves.
- Atlas: accessible from anywhere, automatic backups and scaling, free tier available, but needs internet.

Hinglish:

Local MongoDB apne computer par chalta hai, Atlas cloud par hota hai. Local offline chal jaata hai lekin sirf apne system par. Atlas kahin se bhi access ho jaata hai aur backup wo khud karta hai, bas internet chahiye.

Example:

```
// local
mongodb://127.0.0.1:27017/shopdb

// Atlas
mongodb+srv://user:pass@cluster0.mongodb.net/shopdb
```

Why is it used?

We use local for development and Atlas for deployment, because a hosted app needs a database reachable over the internet.

Interview Answer:

Local MongoDB runs on our own machine and is good for development, while MongoDB Atlas is a managed cloud database used in production because it is accessible from anywhere with backups and scaling included.

Important Point:

- Atlas requires IP whitelisting and a database user.
- Only the connection string changes between the two, not the code.

Point	Local MongoDB	MongoDB Atlas
Location	Own computer	Cloud
Internet	Not needed	Needed
Backup	Manual	Automatic
Access	One machine	Anywhere
Best for	Development	Production and deployment

Q50. Common MongoDB interview mistakes.

Easy English:

Points that candidates often get wrong:

- Saying MongoDB stores JSON, it actually stores BSON.
- Saying MongoDB has no schema at all, it has a flexible schema and Mongoose adds structure.
- Forgetting that MongoDB supports transactions since version 4.0.
- Confusing MongoDB with Mongoose: MongoDB is the database, Mongoose is the library.
- Using index keys or array index as identifiers instead of _id.
- Forgetting await, which returns a pending Promise instead of data.
- Not checking whether findOne returned null before using the result.

Hinglish:

Aksar galtiyan: MongoDB ko JSON database kehna (wo BSON hai), schema bilkul nahi hota kehna (flexible hota hai), transactions nahi hote kehna (ab hote hain), aur MongoDB aur Mongoose ko same samajhna. Await bhoolna aur null check na karna bhi common galti hai.

Example:

```
// Wrong
const user = User.findOne({ email });     // missing await
console.log(user.name);                   // crash

// Right
const user = await User.findOne({ email });
if (!user) return res.status(404).json({ message: "Not found" });
```

Why is it used?

Knowing these points helps avoid common bugs and gives confident, correct answers in the interview.

Interview Answer:

Common mistakes include saying MongoDB stores JSON instead of BSON, claiming it has no schema, forgetting that

FINAL QUICK REVISION

Last-minute notes before the interview

FINAL QUICK REVISION

JavaScript Quick Revision

Core concepts

- JavaScript is single-threaded, dynamically typed and follows the ECMAScript standard.
- Primitive types: string, number, boolean, null, undefined, symbol, bigint. Reference types: object, array, function.
- typeof null returns "object" (a known bug). Use Array.isArray() to detect arrays.
- NaN is of type number and NaN === NaN is false. Check with Number.isNaN().
- Hoisting: var becomes undefined, let and const stay in the Temporal Dead Zone, function declarations are fully hoisted.
- Scope: global, function and block. Lexical scope is the base of closures.
- Closure: an inner function remembering outer variables even after the outer function has returned.

Async

- Event loop order: all synchronous code, then microtasks (Promises), then macrotasks (setTimeout).
- Promise states: pending, fulfilled, rejected. async/await is syntactic sugar over Promises.
- Use try...catch with async/await, and .catch() with plain Promises.
- Promise.all runs in parallel, sequential awaits run one by one.

ES6 features to remember

- Destructuring, spread (...expand), rest (...collect), template literals, optional chaining (?.), nullish coalescing (??), arrow functions, modules.

Hinglish: JavaScript single-threaded hai, async kaam event loop karta hai. Microtask (Promise) hamesha setTimeout se pehle chalta hai. Closure matlab andar wala function bahar ki value yaad rakhta hai. Hoisting me var undefined banta hai aur let/const TDZ me rehte hain.

Node.js Quick Revision

- Node.js is a runtime, not a language and not a framework. It is built on the V8 engine plus libuv.
- It is single-threaded for JavaScript but uses a thread pool for heavy I/O.
- npm manages packages; package.json holds scripts and dependencies; package-lock.json locks exact versions.
- node_modules and .env must be added to .gitignore.
- process.env reads environment variables, loaded from .env using the dotenv package.
- Built-in modules to remember: fs (files), http (server), path (paths), events (EventEmitter), os, crypto.
- CommonJS uses require and module.exports; ES Modules use import and export with "type": "module".
- Avoid Sync methods like readFileSync inside API routes because they block the event loop.
- Node is best for I/O heavy apps, not for CPU heavy work like video encoding.

Hinglish: Node.js ek runtime hai jo V8 par bana hai. JavaScript single thread par chalti hai lekin bhaari kaam libuv ke thread pool ko chala jaata hai. .env aur node_modules kabhi GitHub par push nahi karte. Blocking code poore server ko slow kar deta hai.

Express.js Quick Revision

- Express is a minimal framework built on Node's http module. It gives routing and middleware.
- Basic order in server.js: create app, add middleware (cors, express.json), mount routes, add error middleware last, then app.listen.
- Middleware signature: (req, res, next). Error middleware: (err, req, res, next).
- express.Router() keeps routes modular in separate files.
- Data sources: req.params (URL path), req.query (after ?), req.body (request body).

- Response helpers: res.json(), res.send(), res.status(), res.redirect().
- Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error.
- Authentication = who you are (401). Authorization = what you can do (403).
- JWT has header, payload and signature. Sign it at login, verify it in middleware.
- CORS is a browser rule; use the cors package so React on another port can call the API.

Hinglish: Express Node ke upar bana framework hai. Middleware request aur response ke beech chalta hai aur next() se aage badhta hai. params URL path se, query ? ke baad se, aur body POST/PUT me aati hai. JWT login par banta hai aur middleware me verify hota hai.

React.js Quick Revision

- React is a library for building UIs using components, JSX and a Virtual DOM.
- Props are read-only and come from the parent. State is local and changeable.
- Hooks can only be called at the top level of a component, never inside conditions or loops.

Important hooks

- useState: add state to a component.
- useEffect: side effects such as API calls, timers and subscriptions.
- useRef: access a DOM node or store a value without re-rendering.
- useMemo: cache an expensive calculated value.
- useCallback: cache a function reference.
- useContext: read global data from a Context Provider.

Other must-know points

- Dependency array: none = every render, [] = once, [x] = when x changes.
- Lists need a unique stable key, use the database id, not the array index.
- Conditional rendering uses a ternary or the && operator.
- Controlled components store the input value in state.
- Lifting state up moves shared state to the nearest common parent; Context solves prop drilling.
- React Router gives SPA navigation with BrowserRouter, Routes, Route, Link and useNavigate.
- Reconciliation is the diffing process that updates only the changed DOM nodes.

Hinglish: React component based library hai jo Virtual DOM use karti hai. Props parent se aate hain aur badle nahi ja sakte, state component ka apna data hai. useEffect me khaali array dene par effect sirf ek baar chalta hai. List me key hamesha unique id honi chahiye, index nahi.

MongoDB Quick Revision

- MongoDB is a NoSQL document database storing BSON documents inside collections.
- Every document has a unique _id of type ObjectId, which is indexed by default.

Common commands

- Create: insertOne, insertMany, Model.create
- Read: find, findOne, findById
- Update: updateOne, updateMany, findByIdAndUpdate (use $set)
- Delete: deleteOne, deleteMany, findByIdAndDelete
- Operators: $gt, $gte, $lt, $lte, $ne, $in, $nin, $and, $or, $set, $push, $pull, $inc
- Aggregation stages: $match, $group, $sort, $project, $limit, $skip, $lookup, $unwind. Always put $match first.
- Indexes speed up reads but slow down writes. A unique index prevents duplicate emails.

Mongoose essentials

- Schema defines structure, Model performs operations.
- Validators: required, min, max, minlength, enum, match, unique.
- Hooks: pre("save") to hash a password, post hooks for cleanup.
- populate() resolves references; lean() returns plain fast objects for read-only APIs.
- timestamps: true adds createdAt and updatedAt automatically.
- Connect with mongoose.connect(process.env.MONGO_URI) inside try...catch before app.listen.

Hinglish: MongoDB documents ko BSON me store karta hai aur har document me unique _id hoti hai. Mongoose schema aur validation deta hai. $set se partial update, $push se array me add, $pull se remove. Aggregation me $match sabse pehle rakhein taaki query fast rahe.

Most Important Differences

var vs let vs const

Point	var	let	const
Scope	Function	Block	Block
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No
Hoisting	undefined	TDZ	TDZ
Use	Avoid	Value changes	Default

== vs ===

Point	==	===
Compares	Value only	Value and type
Type conversion	Yes	No
5 == "5"	true	false
Recommended	No	Yes

null vs undefined

Point	null	undefined
Meaning	Intentionally empty	Not assigned
Set by	Developer	JavaScript
typeof	object	undefined

function vs arrow function

Point	Normal function	Arrow function
this	Own this	Parent's this
Hoisting	Declaration hoisted	Not hoisted
new keyword	Allowed	Not allowed
arguments object	Available	Not available

synchronous vs asynchronous

Point	Synchronous	Asynchronous
Waiting	Waits for each line	Does not wait
Blocking	Yes	No
Example	readFileSync	readFile, fetch, await

CommonJS vs ES Modules

Point	CommonJS	ES Modules
Import	require()	import
Export	module.exports	export / export default
Placement	Anywhere	Top level
Browser	No	Yes

PUT vs PATCH

Point	PUT	PATCH
Update	Full replace	Partial
Data sent	Whole object	Changed fields only
Missing fields	May be removed	Unchanged

params vs query vs body

Point	req.params	req.query	req.body
Location	URL path	After ?	Request body
Example	/users/:id	?page=2	JSON data
Purpose	Identify one record	Filter, sort, paginate	Send data

authentication vs authorization

Point	Authentication	Authorization
Question	Who are you?	What can you do?
Order	First	Second
Status code	401	403

props vs state

Point	Props	State
Source	Parent	Own component
Changeable	No	Yes
Purpose	Configure	Track changing data

useMemo vs useCallback

Point	useMemo	useCallback
Returns	A value	A function
Purpose	Skip heavy calculation	Keep same function reference
Used with	Big lists, totals	React.memo child props

SQL vs NoSQL

Point	SQL	NoSQL (MongoDB)
Structure	Tables, rows	Collections, documents
Schema	Fixed	Flexible
Relations	JOIN	Embed or $lookup
Scaling	Vertical	Horizontal

MongoDB vs Mongoose

Point	MongoDB	Mongoose
What it is	NoSQL database	ODM library for Node
Schema	Flexible, optional	Defined and enforced
Validation	Limited	Built-in validators
Extra features	-	Hooks, populate, lean

embedding vs referencing

Point	Embedding	Referencing
Data location	Same document	Separate collection
Read speed	Fast, one query	Needs populate or $lookup
Best for	One-to-few	One-to-many
Risk	16 MB limit	Extra queries

Interview Speaking Practice

Read these out loud a few times. Speak the English line in the interview, and use the Hinglish line only to make sure you understand the meaning.

1. Interviewer: "What is JavaScript?"

Answer: "JavaScript is a high-level, dynamically typed programming language used to make web pages interactive. It runs in the browser and also on the server through Node.js, so we can build both frontend and backend with one language."

Hinglish: "JavaScript ek high-level programming language hai jo website ko interactive banati hai. Ye browser me bhi chalti hai aur Node.js ke through server par bhi, isliye frontend aur backend dono ek hi language me ban jaate hain."

2. Interviewer: "What is a closure?"

Answer: "A closure is a function that remembers the variables of its outer function even after the outer function has finished executing. It is used for data privacy and to keep values between calls."

Hinglish: "Closure wo function hai jo apne bahar wale function ki value yaad rakhta hai, chahe wo function khatam ho chuka ho. Isse private data aur counter jaise cases handle hote hain."

3. Interviewer: "What is the event loop?"

Answer: "The event loop allows single-threaded JavaScript to handle asynchronous tasks. It checks whether the call stack is empty and then pushes pending callbacks from the microtask and callback queues into the stack. Promises run before setTimeout."

Hinglish: "Event loop ki wajah se single-threaded JavaScript async kaam kar paati hai. Jab call stack khaali hota hai tab queue se callback uthakar chalata hai. Promise wale microtask setTimeout se pehle chalte hain."

4. Interviewer: "What is Node.js?"

Answer: "Node.js is a JavaScript runtime environment based on the V8 engine. It allows us to run JavaScript outside the browser, mainly for backend development, APIs and real-time applications."

Hinglish: "Node.js ek JavaScript runtime environment hai jo V8 engine par bana hai. Isse hum JavaScript ko browser ke bahar chala sakte hain, mainly backend, API aur real-time applications ke liye."

5. Interviewer: "What is Express.js?"

Answer: "Express.js is a minimal web framework built on top of Node.js. It simplifies routing, middleware and request handling, which makes building REST APIs much faster."

Hinglish: "Express.js Node.js ke upar bana ek halka framework hai. Ye routing aur middleware ko simple bana deta hai, jisse REST API banana bahut fast ho jaata hai."

6. Interviewer: "What is middleware?"

Answer: "Middleware is a function that runs between the request and the response. It has access to req, res and next, and we use it for body parsing, authentication, logging and error handling."

Hinglish: "Middleware wo function hai jo request aur response ke beech chalta hai. Iske paas req, res aur next hote hain, aur ye authentication, logging aur error handling ke liye use hota hai."

7. Interviewer: "What is JWT and why do we use it?"

Answer: "JWT is a JSON Web Token made of header, payload and signature. The server signs it with a secret key at login and the client sends it back with each request, so authentication stays stateless and easy to scale."

Hinglish: "JWT ek token hai jisme header, payload aur signature hote hain. Login par server ise secret key se sign karta hai aur client har request ke saath bhejta hai, isliye server ko session store nahi karna padta."

8. Interviewer: "What is React?"

Answer: "React is a JavaScript library for building user interfaces. It is component based, uses a Virtual DOM for efficient updates, and follows a declarative approach where the UI updates automatically when state changes."

Hinglish: "React ek JavaScript library hai jo user interface banane ke liye hai. Ye components me kaam karti hai, Virtual DOM use karti hai aur state badalne par UI khud update kar deti hai."

9. Interviewer: "Difference between props and state?"

Answer: "Props are read-only data passed from a parent component to a child, while state is data owned by the component itself and can be changed using its setter. Both trigger a re-render when they change."

Hinglish: "Props parent se child ko milte hain aur read-only hote hain, jabki state component ka apna data hai jo badal sakta hai. Dono badalne par component dobara render hota hai."

10. Interviewer: "What is useEffect used for?"

Answer: "useEffect is used for side effects like API calls, timers and subscriptions. It runs after render, and the dependency array controls when it runs again. An empty array means it runs only once after mount."

Hinglish: "useEffect side effects ke liye hota hai jaise API call ya timer. Ye render ke baad chalta hai aur dependency array batata hai kab dobara chalega. Khaali array ka matlab sirf ek baar chalega."

11. Interviewer: "What is MongoDB?"

Answer: "MongoDB is a NoSQL document database that stores data in flexible JSON-like BSON documents instead of tables. It has no fixed schema and fits naturally with JavaScript, which is why it is used in the MERN stack."

Hinglish: "MongoDB ek NoSQL document database hai jo data ko table ki jagah JSON jaise documents me rakhta hai. Iska schema flexible hai aur JavaScript ke saath achhe se fit hota hai."

12. Interviewer: "What is Mongoose?"

Answer: "Mongoose is an Object Data Modeling library for MongoDB and Node.js. It adds schemas, models, validation, middleware and populate, which gives structure and safety to a schema-less database."

Hinglish: "Mongoose ek ODM library hai jo MongoDB aur Node ke beech kaam karti hai. Ye schema, validation, hooks aur populate deti hai, jisse database ka data hamesha sahi structure me rehta hai."

13. Interviewer: "Explain the MERN stack in one line."

Answer: "MERN stands for MongoDB, Express.js, React.js and Node.js. React builds the frontend, Express and Node handle the backend and APIs, and MongoDB stores the data, all using JavaScript."

Hinglish: "MERN ka matlab MongoDB, Express, React aur Node hai. React frontend banata hai, Express aur Node backend aur API sambhalte hain, aur MongoDB data store karta hai, aur ye sab JavaScript me hota hai."

14. Interviewer: "How does data flow in a MERN application?"

Answer: "The React component calls the API using fetch or axios. Express receives the request, middleware handles authentication and parsing, the controller uses a Mongoose model to read or write in MongoDB, and the JSON response goes back to React, which updates the state and re-renders the UI."

Hinglish: "React component fetch ya axios se API call karta hai. Express request leta hai, middleware authentication karta hai, controller Mongoose model se MongoDB me kaam karta hai, aur JSON response React ko wapas jaata hai jahan state update hoke UI render ho jaata hai."

All the best for your interview, Deepak. Revise the difference tables and the speaking practice section on the morning of the interview.